EE204 Computer Architecture

Pipeline: Hazards

Nouman M Durrani

Courtesy: Prof. Hong Jiang

Courtesy of Prof. Yifeng Zhu, U. of Maine

Fall, 2026

Pipelining Outline

- **Introduction**
  - Defining Pipelining
  - Pipelining Instructions
- **Hazards**
  - Structural hazards ⇦
  - Data Hazards
  - Control Hazards
- **Performance**
- **Controller implementation**

Pipelining is an implementation technique whereby multiple instructions are overlapped in execution

Pipeline Hazards

- **Where one instruction cannot immediately follow another**
- **Types of hazards**
  - **Structural hazards - attempt to use the same resource by two or more instructions**
  - **Control hazards - attempt to make branching decisions before branch condition is evaluated**
  - **Data hazards - attempt to use data before it is ready**
- **Can always resolve hazards by waiting**

Structural Hazards

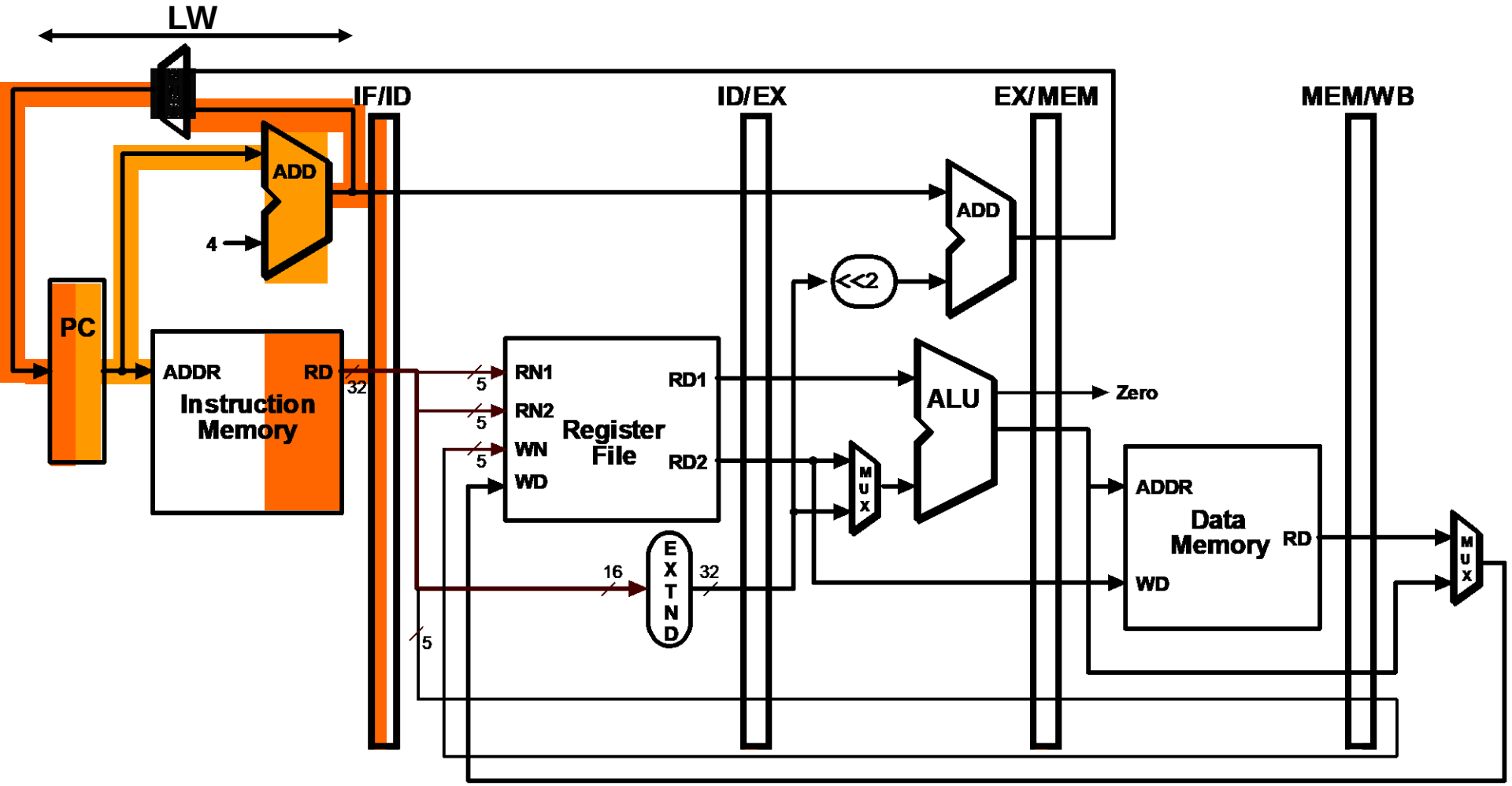
- **Attempt to use the same resource by two or more instructions at the same time**
- **Example: Single Memory for instructions and data**
  - **Accessed by IF stage**
  - **Accessed at same time by MEM stage**
- **Solutions**
  - **Delay the second access by one clock cycle, OR**
  - **Provide separate memories for instructions & data**
    - » **This is what the book does**
    - » **This is called a "Harvard Architecture"**
    - » **Real pipelined processors have separate caches**

Pipelined Example - Executing Multiple Instructions

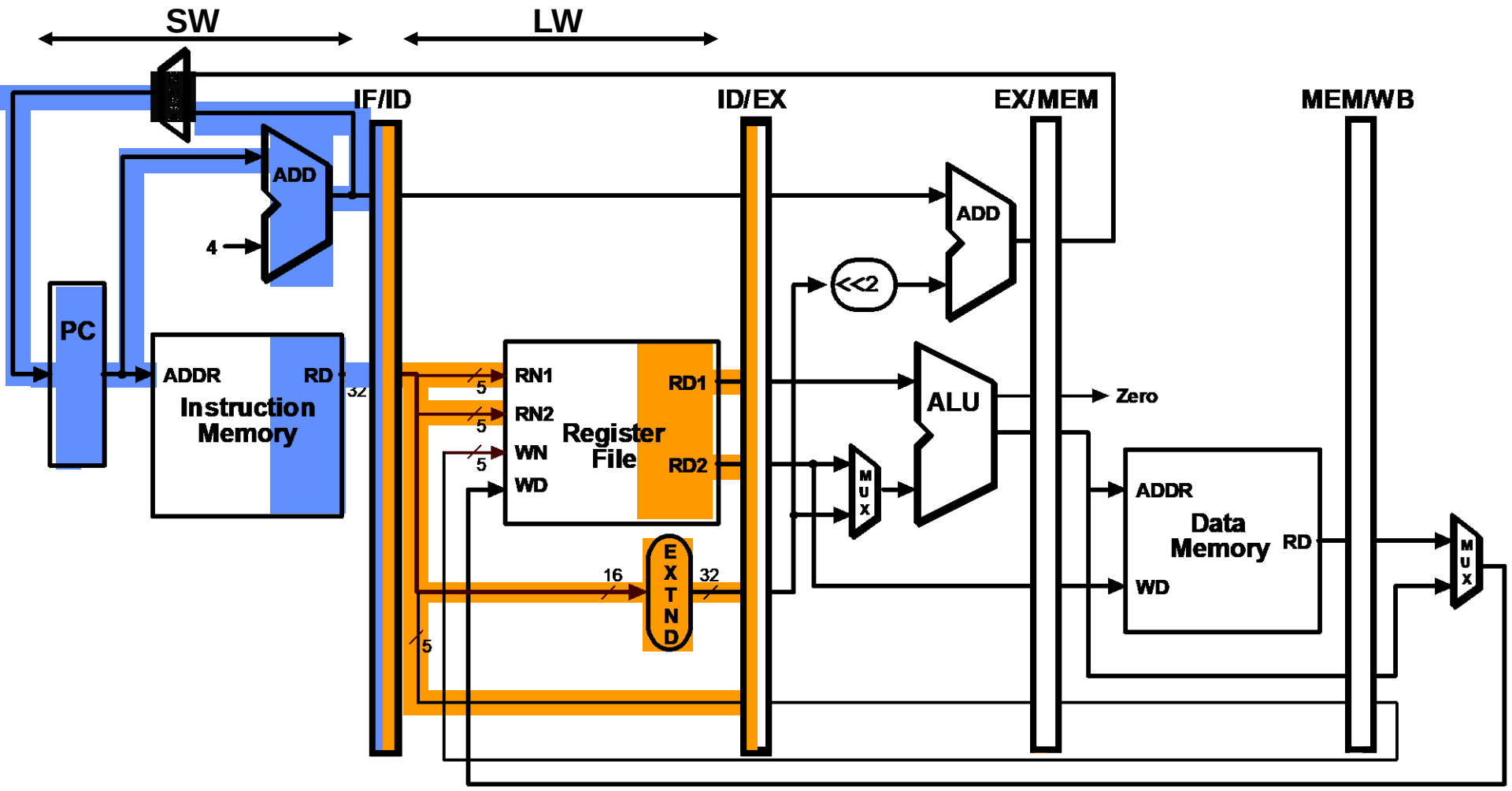
- **Consider the following instruction sequence:**

```
lw $r0, 10($r1)
sw $sr3, 20($r4)
add $r5, $r6, $r7
sub $r8, $r9, $r10
```

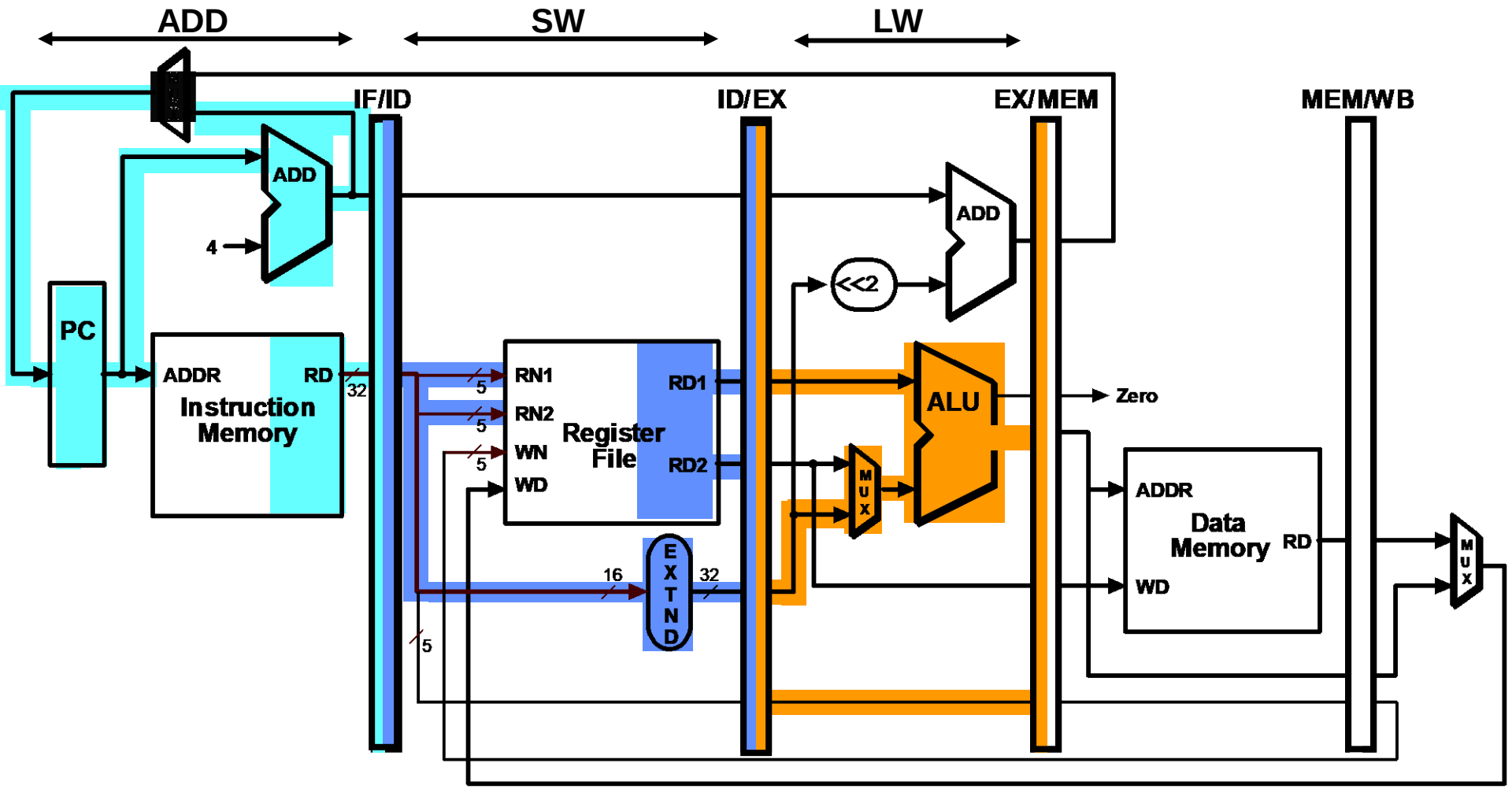
Executing Multiple Instructions
Clock Cycle 1



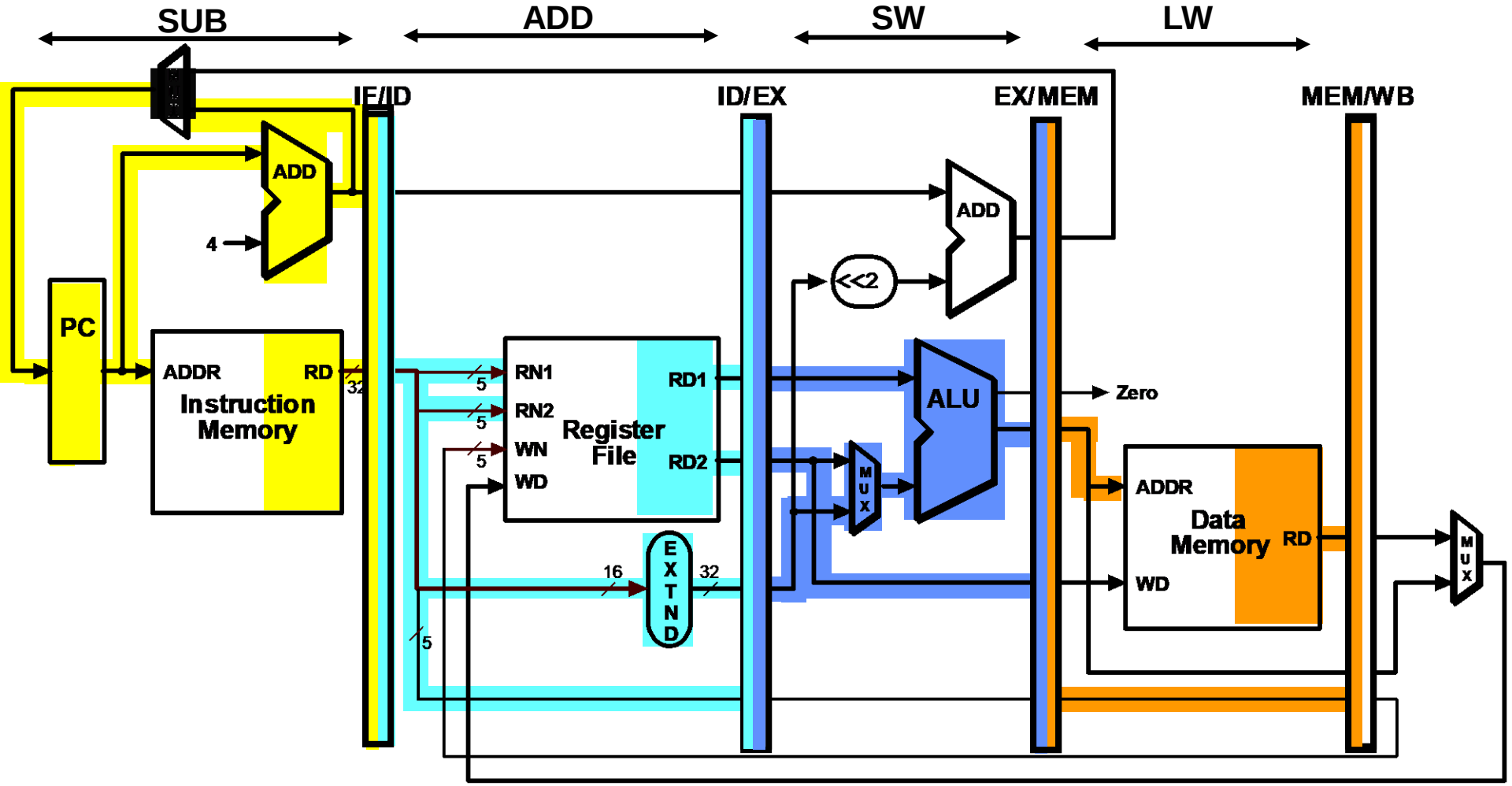
Executing Multiple Instructions
Clock Cycle 2



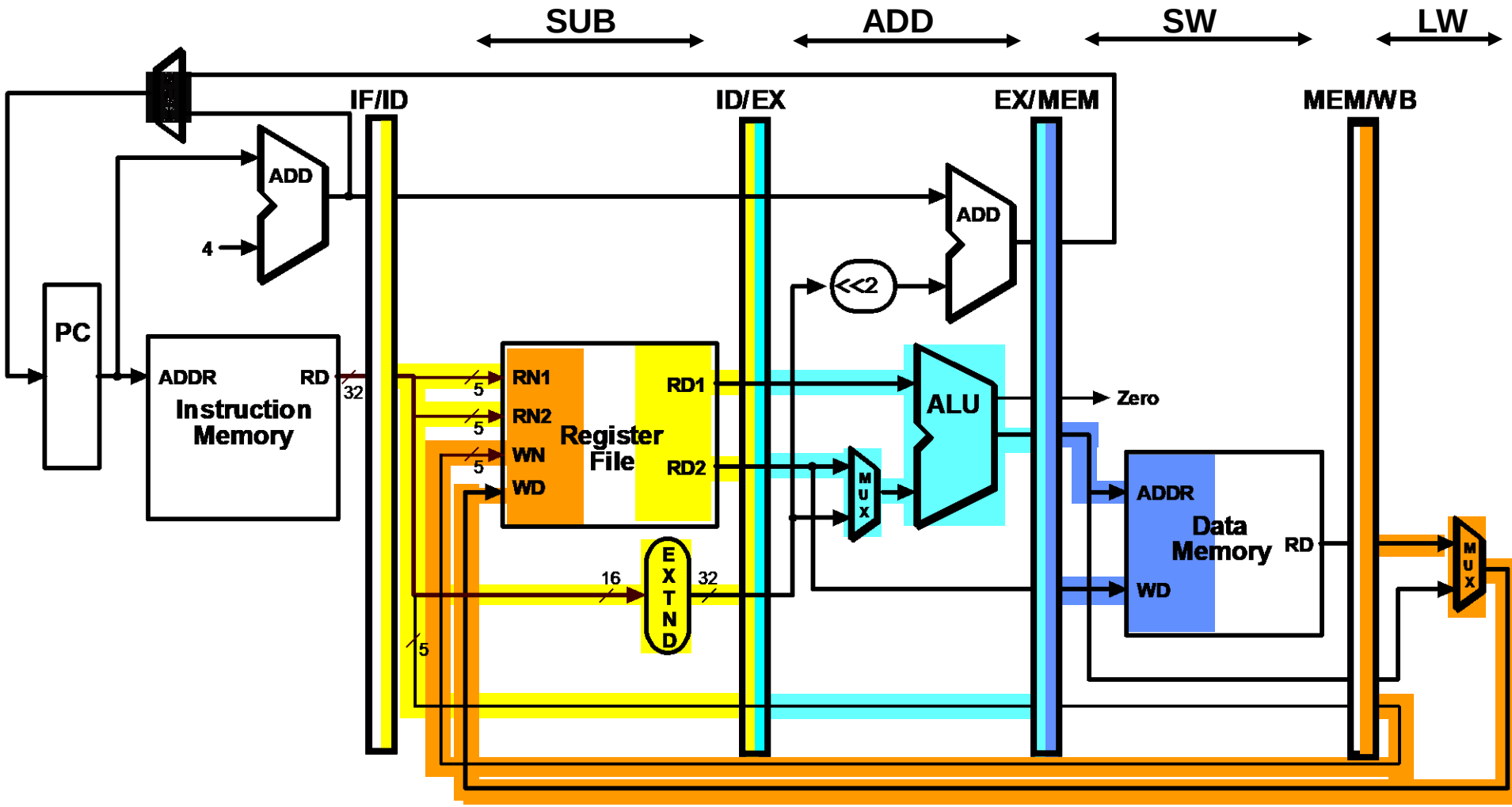
Executing Multiple Instructions
Clock Cycle 3



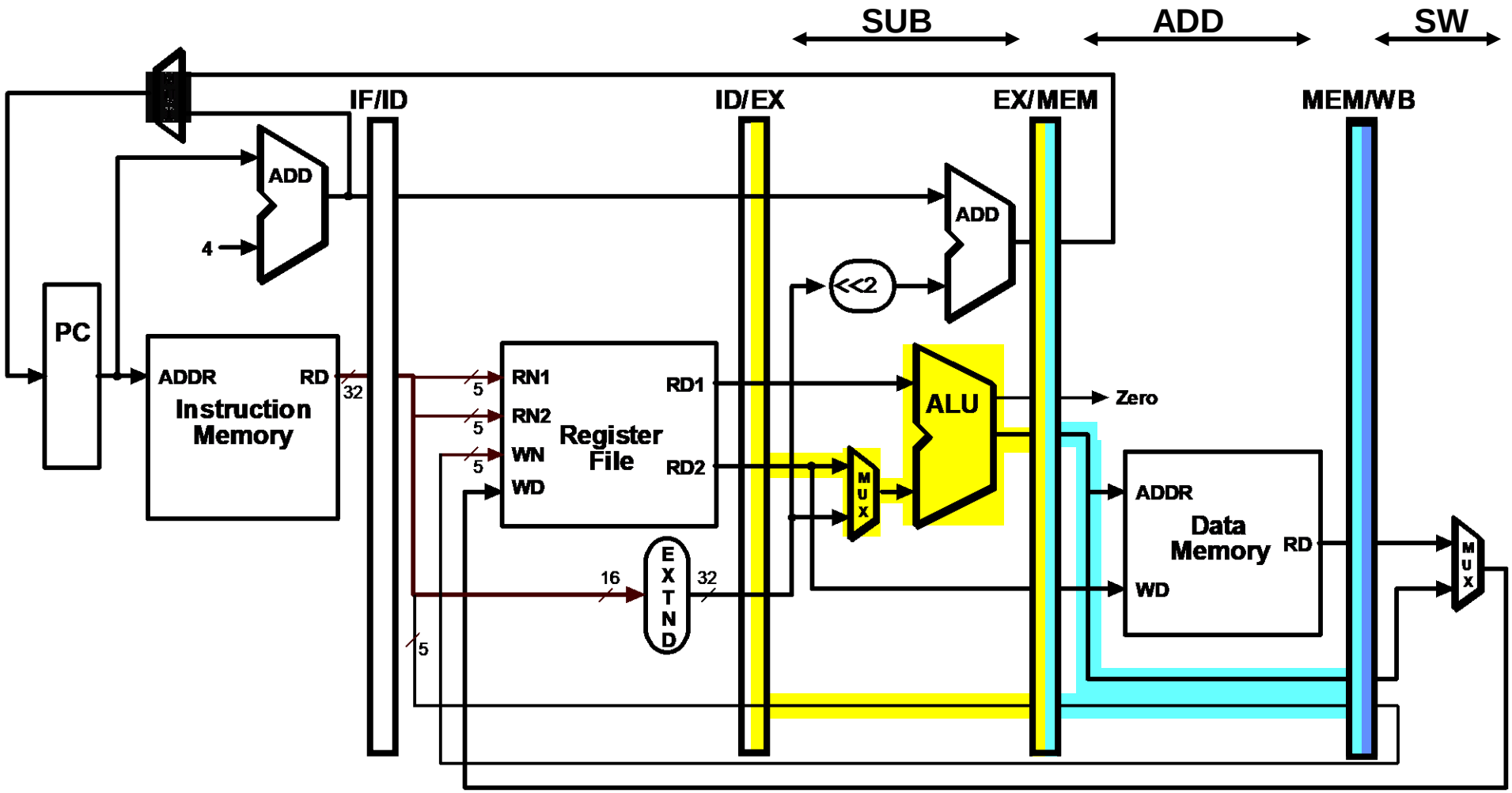
Executing Multiple Instructions
Clock Cycle 4



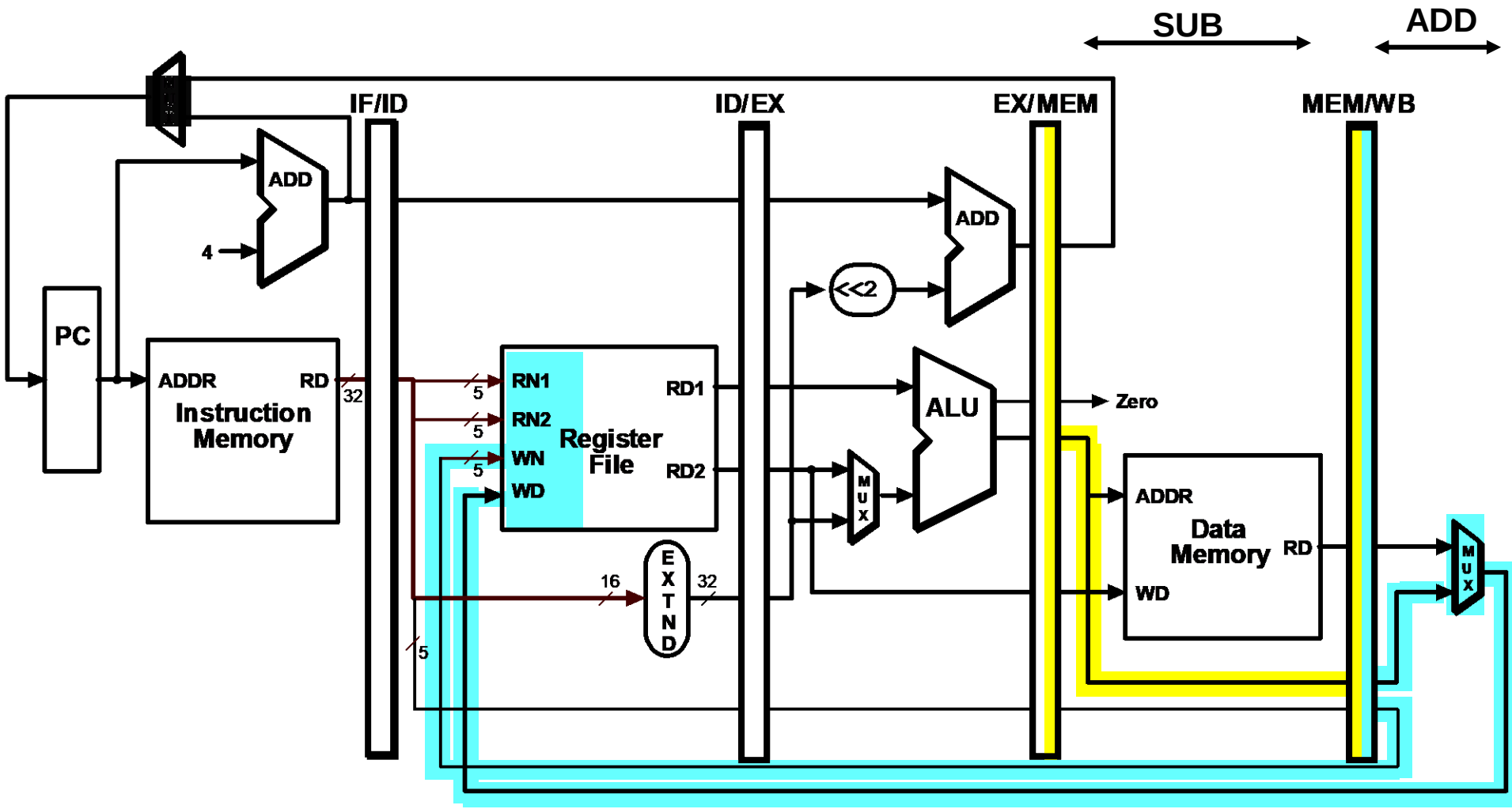
Executing Multiple Instructions
Clock Cycle 5



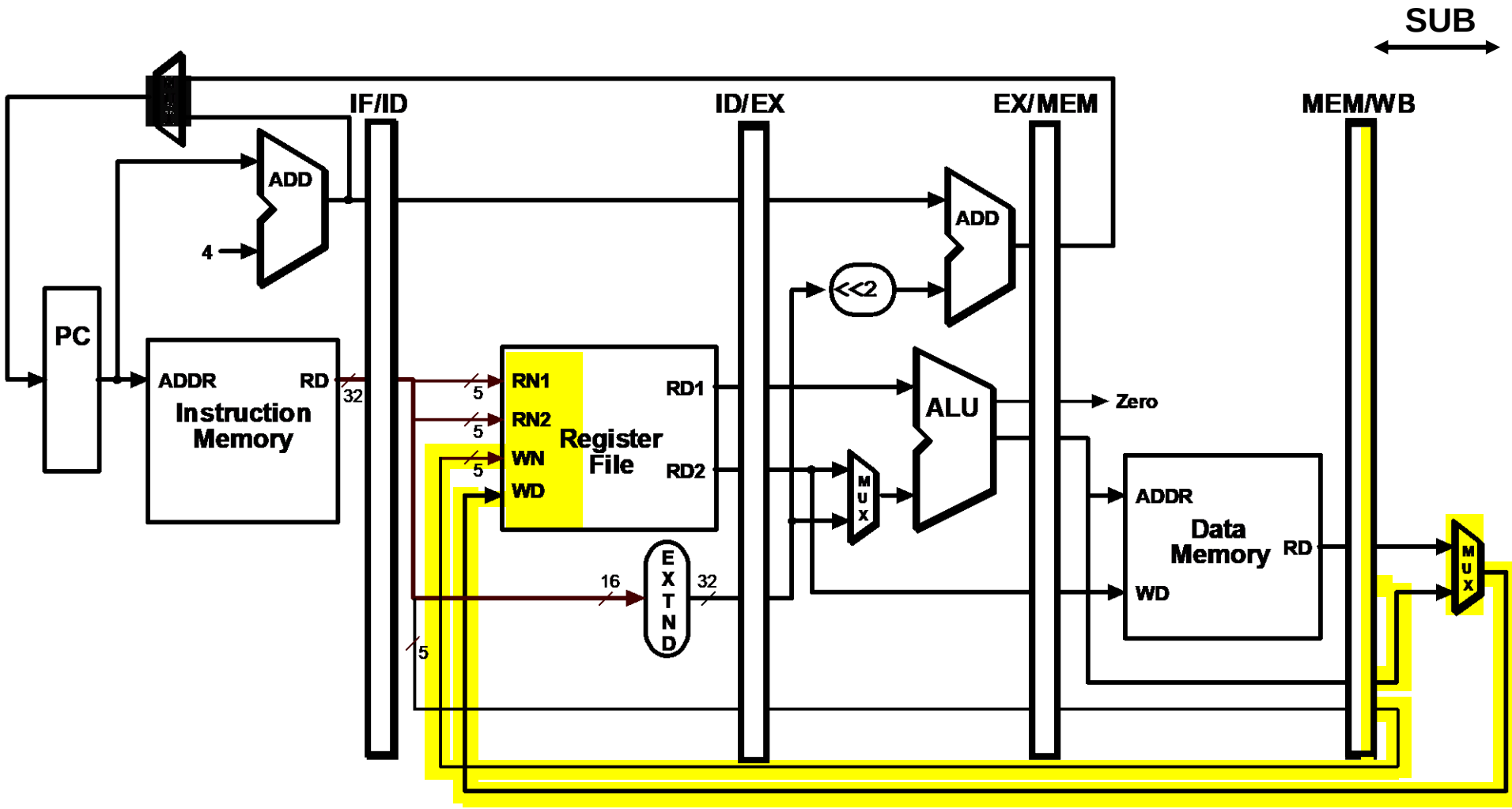
Executing Multiple Instructions
Clock Cycle 6



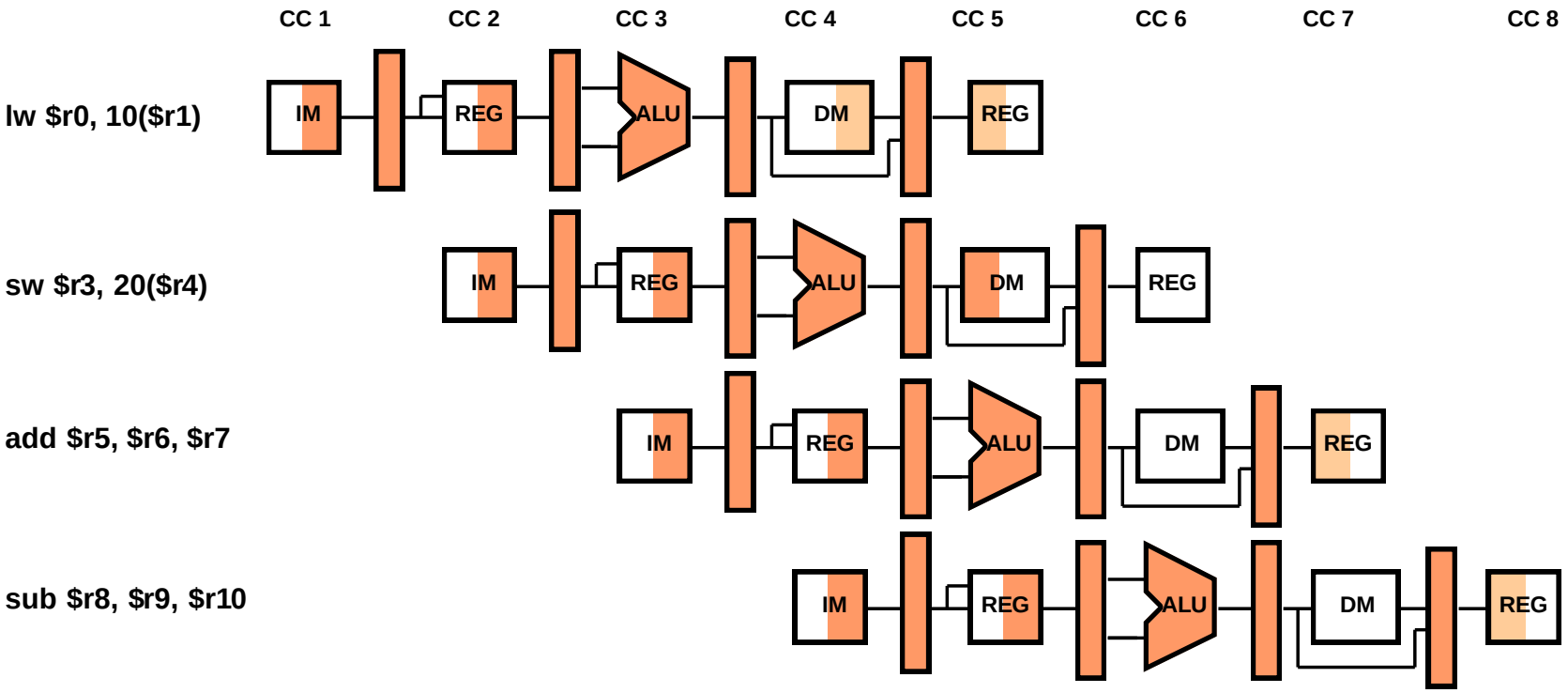
Executing Multiple Instructions
Clock Cycle 7



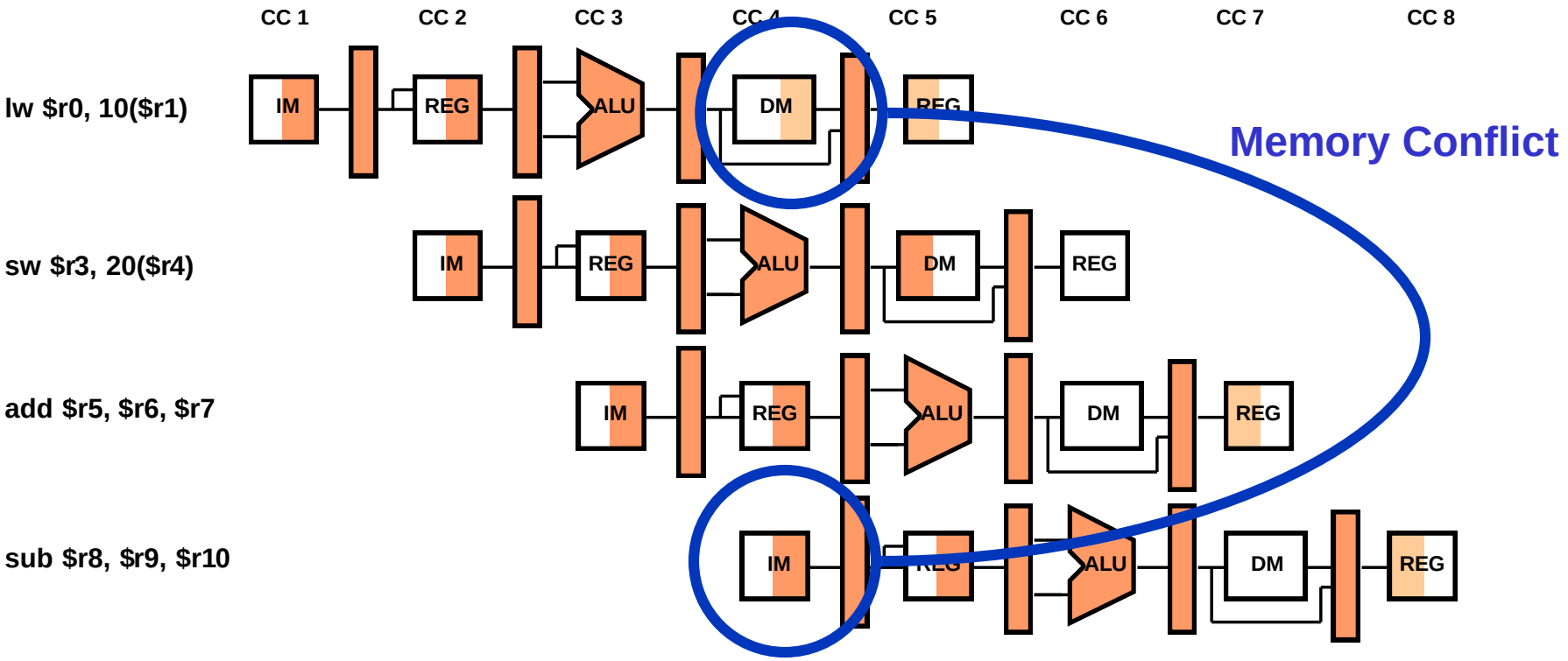
Executing Multiple Instructions
Clock Cycle 8



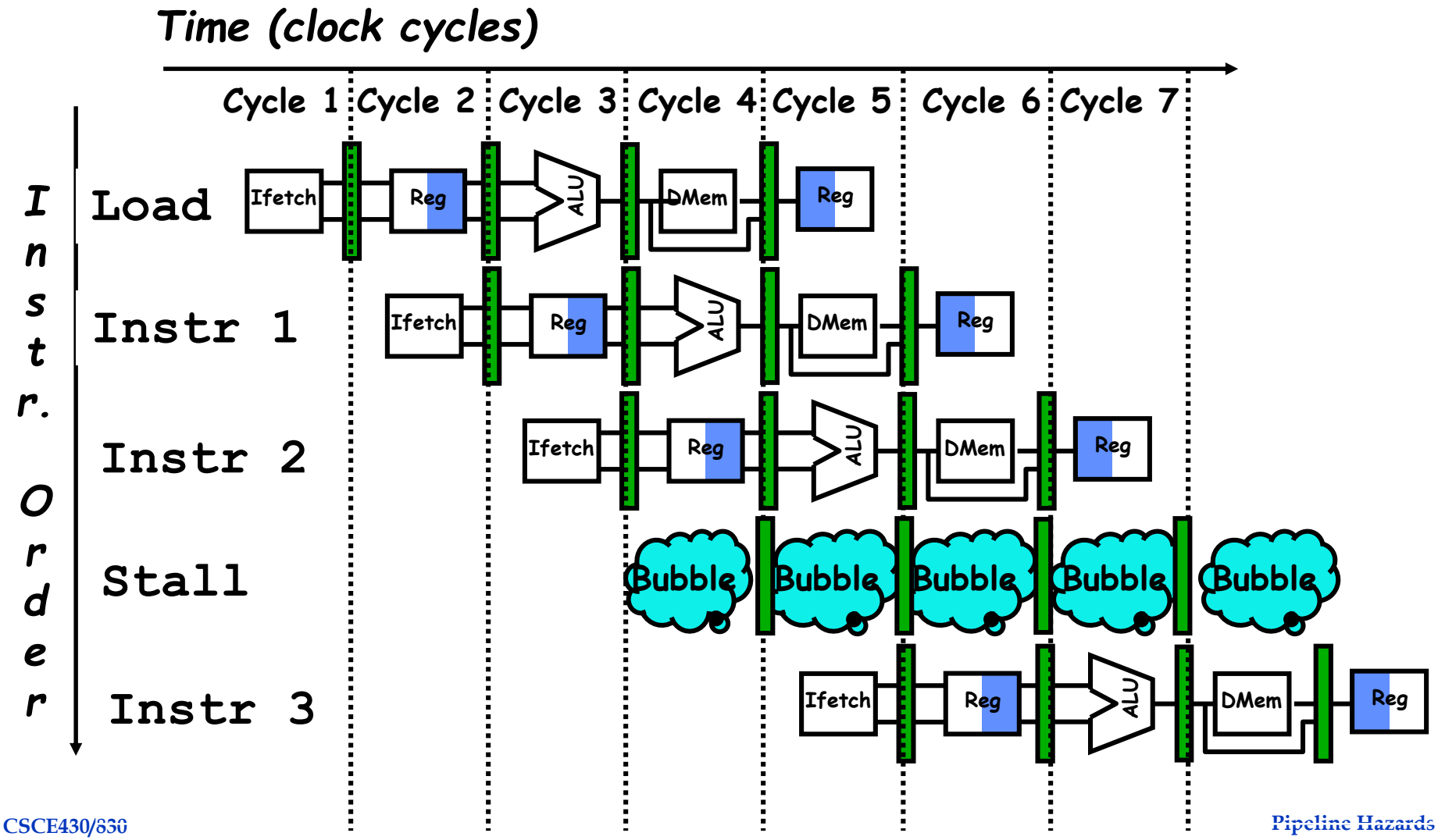
Alternative View - Multicycle Diagram



Alternative View - Multicycle Diagram



One Memory Port Structural Hazards



Structural Hazards

Some common Structural Hazards:

- **Memory:**
  - **we've already mentioned this one.**
- **Floating point:**
  - **Since many floating point instructions require many cycles, it's easy for them to interfere with each other.**
- **Starting up more of one type of instruction than there are resources.**
  - **For instance, the PA-8600 can support two ALU + two load/store instructions per cycle - that's how much hardware it has available.**

Structural Hazards

Dealing with Structural Hazards

Stall

- low cost, simple
- Increases CPI
- use for rare case since stalling has performance effect

Pipeline hardware resource

- useful for multi-cycle resources
- good performance
- sometimes complex e.g., RAM

Replicate resource

- good performance
- increases cost (+ maybe interconnect delay)
- useful for cheap or divisible resources

Structural Hazards

- **Structural hazards are reduced with these rules:**
  - **Each instruction uses a resource at most once**
  - **Always use the resource in the same pipeline stage**
  - **Use the resource for one cycle only**
- **Many RISC ISAs are designed with this in mind**
- **Sometimes very difficult to do this.**
  - **For example, memory of necessity is used in the IF and MEM stages.**

Structural Hazards

We want to compare the performance of two machines. Which machine is faster?

- **Machine A: Dual ported memory - so there are no memory stalls**
- **Machine B: Single ported memory, but its pipelined implementation has a clock rate that is 1.05 times faster**

Assume:

- **Ideal CPI = 1 for both**
- **Loads are 40% of instructions executed**

Speed Up Equations for Pipelining

$$\text{Speedup from pipelining} = \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

$$= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}$$

$$= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}$$

$$CPI_{pipelined} = \text{Ideal CPI} + \text{Average Stall cycles per Inst}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{unpipelined}}{\text{Cycle Time}_{pipelined}}$$

For simple RISC pipeline, CPI = 1:

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{unpipelined}}{\text{Cycle Time}_{pipelined}}$$

Suppose we have a **5-stage pipeline:**

$$IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$$

Therefore:

$$\text{Pipeline depth} = 5$$

Without pipelining

The entire instruction passes through all 5 stages **one after another**.

So one instruction requires:

$$5 \text{ cycles}$$

Therefore:

$$CPI_{unpipelined} = 5$$

In general:

$$\boxed{CPI_{unpipelined} = \text{Pipeline depth}}$$

Structural Hazards

We want to compare the performance of two machines. Which machine is faster?

- **Machine A: Dual ported memory - so there are no memory stalls**
- **Machine B: Single ported memory, but its pipelined implementation has a 1.05 times faster clock rate**

Assume:

- **Ideal CPI = 1 for both**
- **Loads are 40% of instructions executed**

Structural Hazards

We want to compare the performance of two machines. Which machine is faster?

- **Machine A: Dual ported memory - so there are no memory stalls**
- **Machine B: Single ported memory, but its pipelined implementation has a 1.05 times faster clock rate**

Assume:

- **Ideal CPI = 1 for both**
- **Loads are 40% of instructions executed**

$$\text{SpeedUp}_A = \text{Pipeline Depth}/(1 + 0) \times (\text{clock}_{unpipe}/\text{clock}_{pipe})$$
$$= \text{Pipeline Depth}$$
$$\text{SpeedUp}_B = \text{Pipeline Depth}/(1 + 0.4 \times 1) \times (\text{clock}_{unpipe}/(\text{clock}_{unpipe} / 1.05)$$
$$= (\text{Pipeline Depth}/1.4) \times 1.05$$
$$= 0.75 \times \text{Pipeline Depth}$$

SpeedUp_A / SpeedUp_B = Pipeline Depth / (0.75 x Pipeline Depth) = 1.33

- **Machine A is 1.33 times faster**

Pipelining Summary

- **Speed Up <= Pipeline Depth; if ideal CPI is 1, then:**

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle Unpipelined}}{\text{Clock Cycle Pipelined}}$$

- **Hazards limit performance on computers:**
  - **Structural: need more HW resources**
  - **Data (RAW,WAR,WAW)**
  - **Control**

Review

Speedup of pipeline

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle Unpipelined}}{\text{Clock Cycle Pipelined}}$$

Pipelining Outline

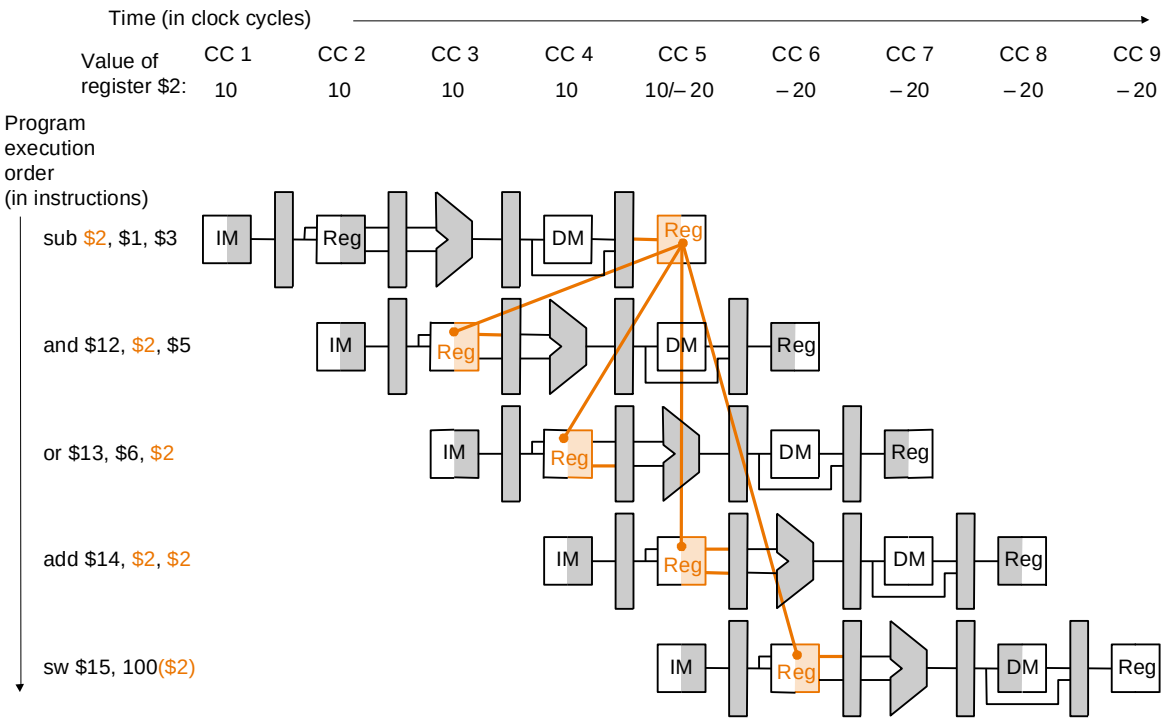
- **Introduction**
  - Defining Pipelining
  - Pipelining Instructions
- **Hazards**
  - Structural hazards
  - Data Hazards ⇦
  - Control Hazards
- **Performance**
- **Controller implementation**

Pipeline Hazards

- **Where one instruction cannot immediately follow another**
- **Types of hazards**
  - **Structural hazards - attempt to use same resource twice**
  - **Control hazards - attempt to make decision before condition is evaluated**
  - **Data hazards - attempt to use data before it is ready**
- **Can always resolve hazards by waiting**

Data Hazards

- **Data hazards occur when data is used before it is ready**



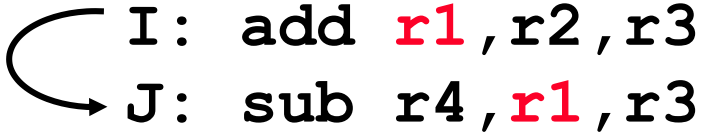
The use of the result of the SUB instruction in the next three instructions causes a data hazard, since the register $2 is not written until after those instructions read it.

Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Read After Write (RAW)

$Instr_J$ tries to read operand before $Instr_I$ writes it



- **Caused by a "Dependence" (in compiler nomenclature). This hazard results from an actual need for communication.**

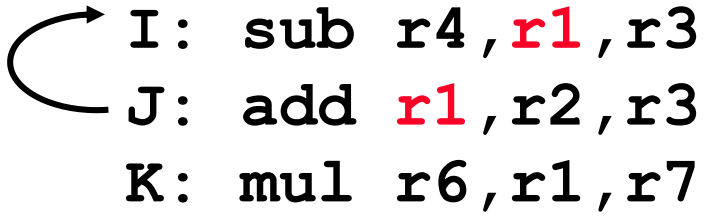
Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Write After Read (WAR)

$Instr_J$ tries to write operand *before* $Instr_I$ reads i

- **Gets wrong operand**



- **Called an "anti-dependence" by compiler writers. This results from reuse of the name "r1".**

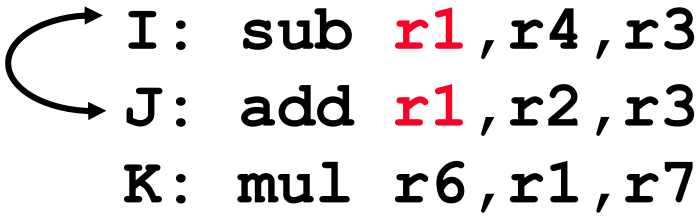
- **Can't happen in MIPS 5 stage pipeline because:**
  - **All instructions take 5 stages, and**
  - **Reads are always in stage 2, and**
  - **Writes are always in stage 5**

Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Write After Write (WAW)

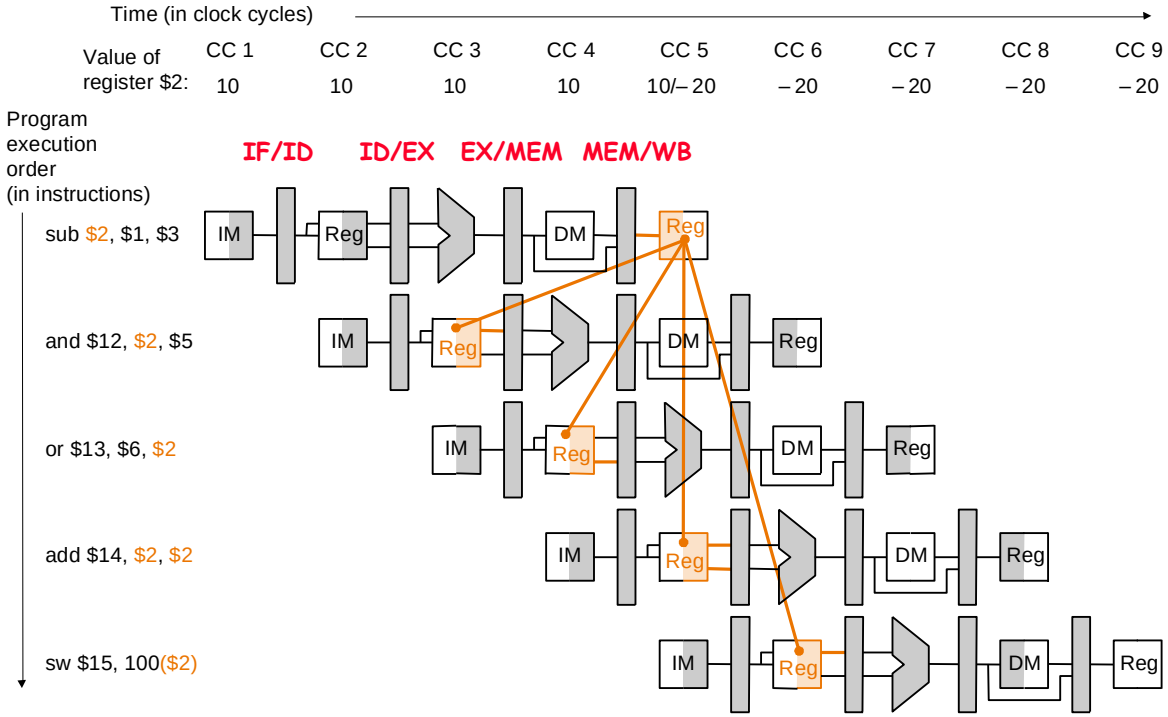
$Instr_J$ tries to write operand *before* $Instr_I$ writes it
- **Leaves wrong result ($Instr_I$ not $Instr_J$)**



- **Called an "output dependence" by compiler writers This also results from the reuse of name "r1".**
- **Can't happen in MIPS 5 stage pipeline because:**
  - **All instructions take 5 stages, and**
  - **Writes are always in stage 5**
- **Will see WAR and WAW later in more complicated pipes**

Data Hazard Detection in MIPS (1)

Read after Write



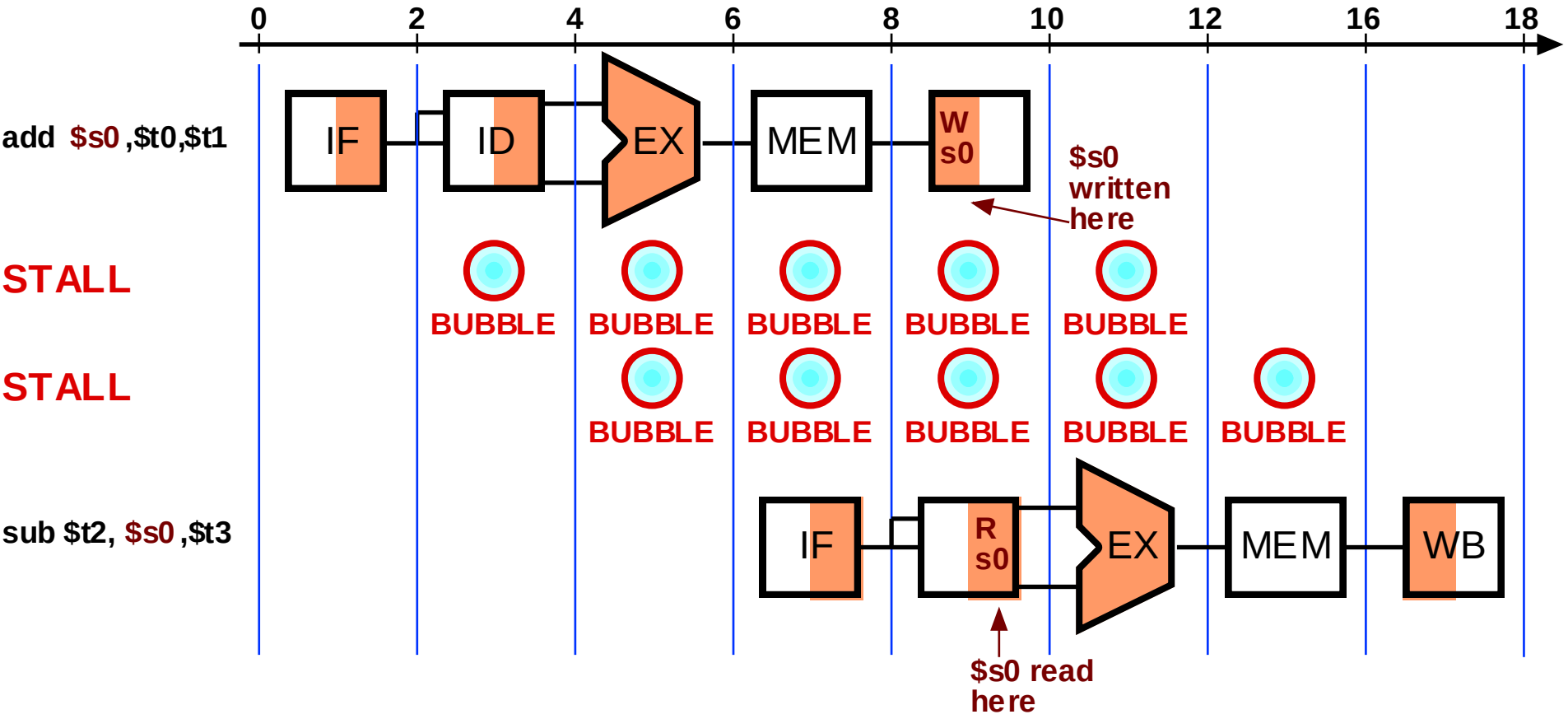
1a: EX/MEM.RegisterRd = ID/EX.RegisterRs
1b: EX/MEM.RegisterRd = ID/EX.RegisterRt
} EX hazard

2a: MEM/WB.RegisterRd = ID/EX.RegisterRs
2b: MEM/WB.RegisterRd = ID/EX.RegisterRt
} MEM hazard

Data Hazards

- **Solutions for Data Hazards**
  - **Stalling**
  - **Forwarding:**
    - » **connect new value directly to next stage**
  - **Reordering**

Data Hazard - Stalling



Data Hazards - Stalling

Simple Solution to RAW

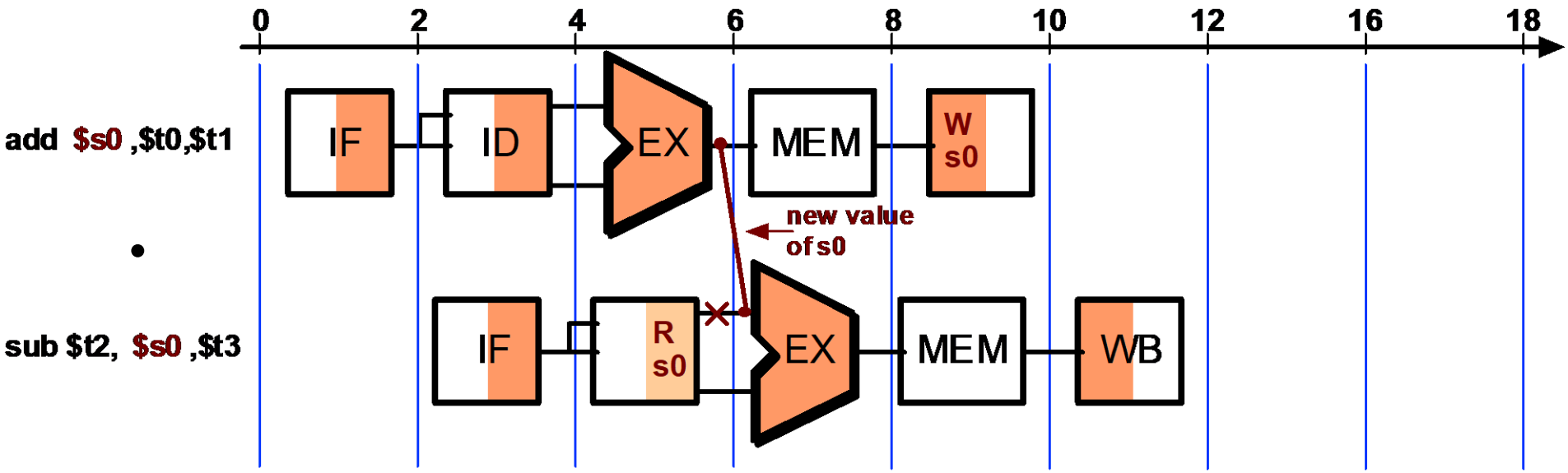
- Hardware detects RAW and stalls
- Assumes register written then read each cycle
  + low cost to implement, simple
  -- reduces IPC
- Try to minimize stalls

Minimizing RAW stalls

- Bypass/forward/shortcircuit (We will use the word “forward”)
- Use data before it is in the register
  + reduces/avoids stalls
  -- complex
- Crucial for common RAW hazards

Data Hazards - Forwarding

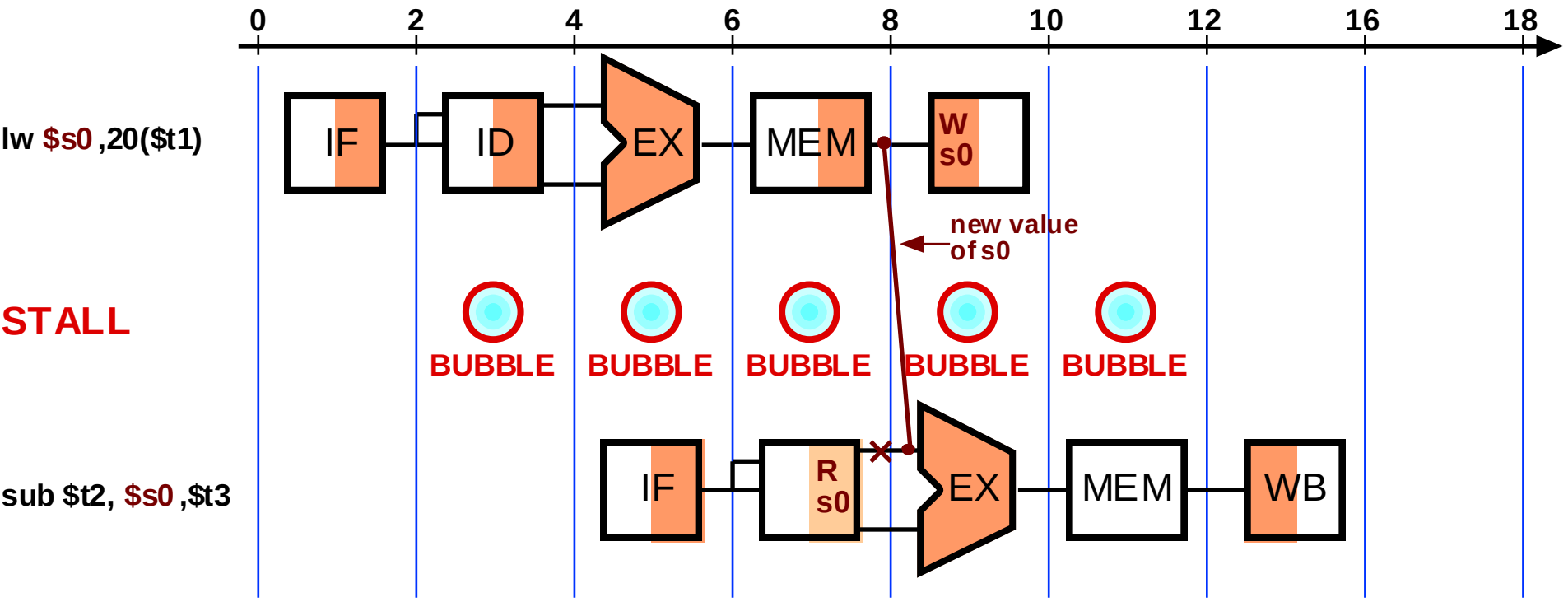
- Key idea: connect new value directly to next stage
- Still read s0, but ignore in favor of new result



- Problem: what about load instructions?

Data Hazards - Forwarding

- STALL still required for load - data avail. after MEM
- MIPS architecture calls this delayed load, initial implementations required compiler to deal with this



Data Hazards

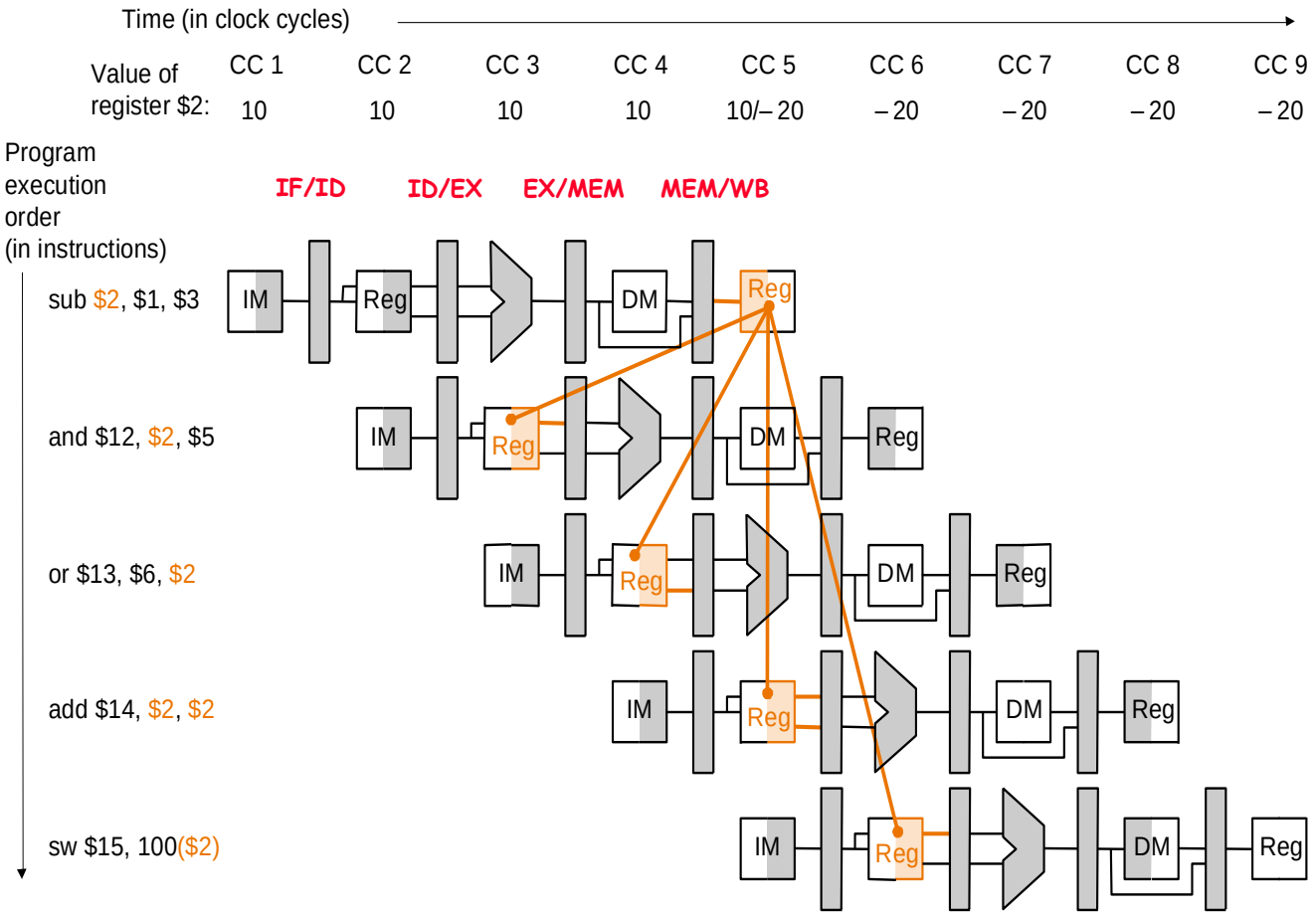
This is another representation of the stall.

LW R1, 0(R2)	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

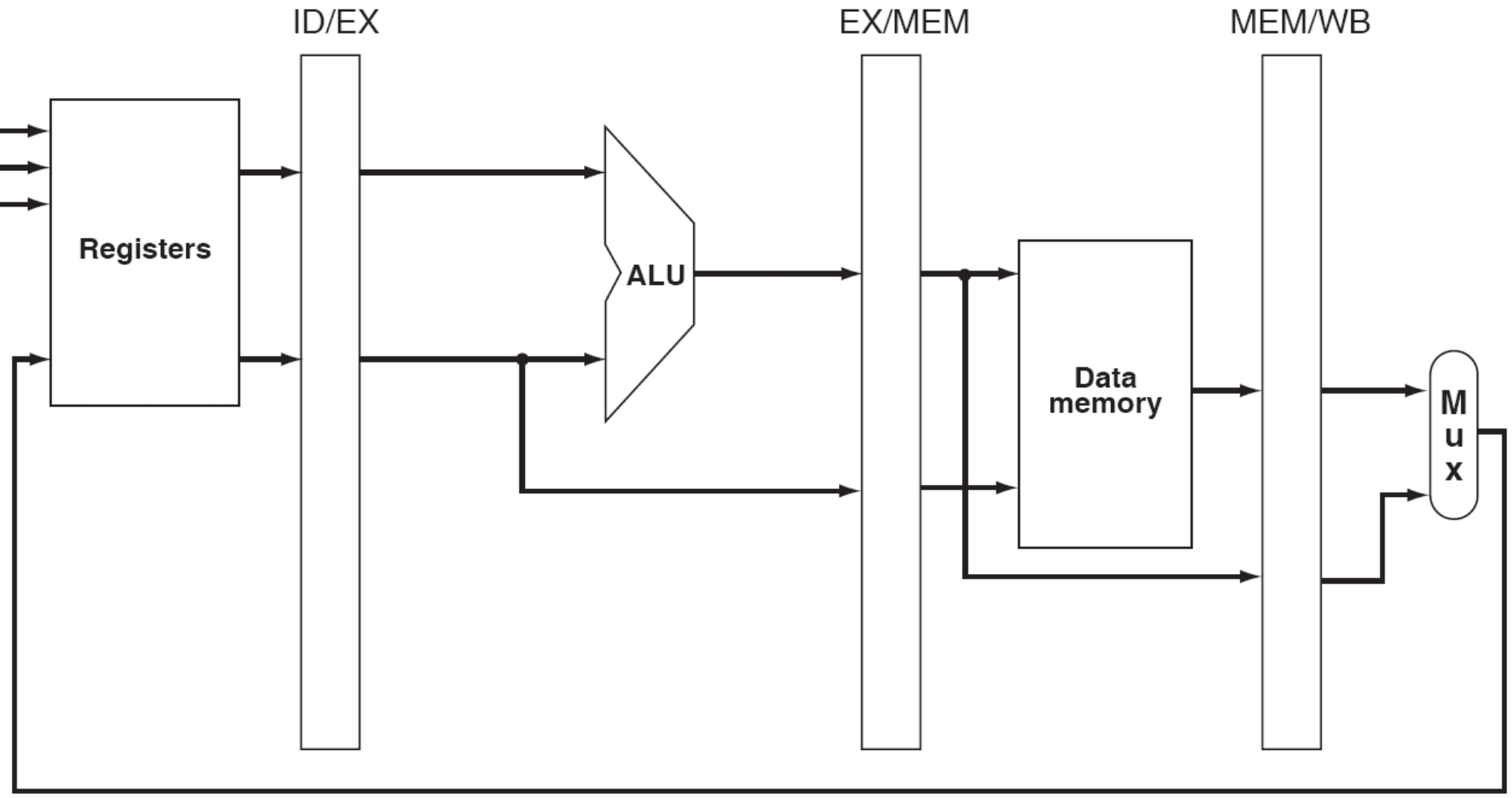
Forwarding

Key idea: connect data internally before it's stored



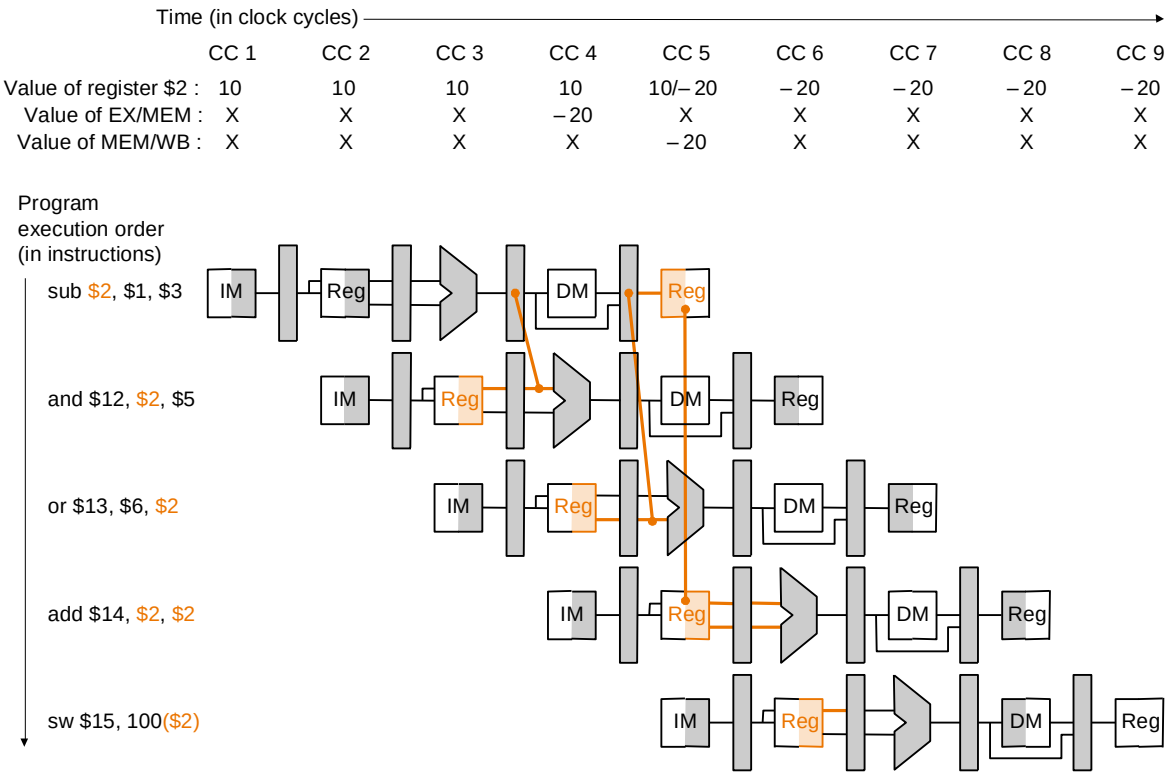
How would you design the forwarding?

No Forwarding



Data Hazard Solution: Forwarding

- **Key idea: connect data internally before it's stored**



Assumption:

- **The register file forwards values that are read and written during the same cycle.**

Data Hazard Summary

- **Three types of data hazards**
  - **RAW (MIPS)**
  - **WAW (not in MIPS)**
  - **WAR (not in MIPS)**
- **Solution to RAW in MIPS**
  - **Stall**
  - **Forwarding**
    - » **Detection & Control**
      - **EX hazard**
      - **MEM hazard**
    - » **A stall is needed if read a register after a load instruction that writes the same register.**
  - **Reordering**

Review

Speedup of pipeline

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle Unpipelined}}{\text{Clock Cycle Pipelined}}$$

Pipelining Outline

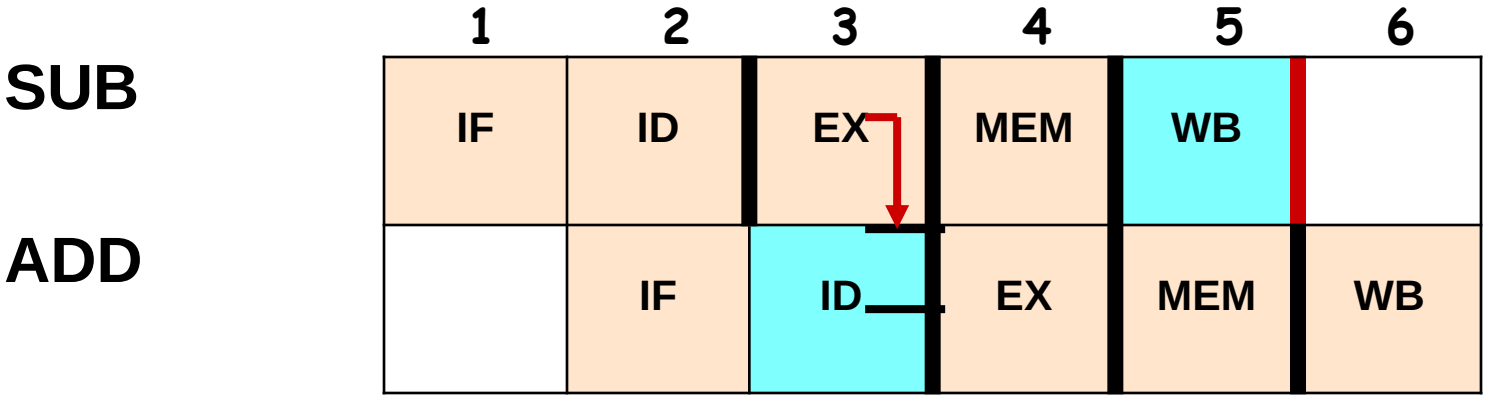
- **Introduction**
  - Defining Pipelining
  - Pipelining Instructions
- **Hazards**
  - Structural hazards
  - Data Hazards ⇦
  - Control Hazards
- **Performance**
- **Controller implementation**

Data Hazard Review

- **Three types of data hazards**
  - **RAW (in MIPS and all others)**
  - **WAW (not in MIPS but many others)**
  - **WAR (not in MIPS but many others)**
- **Forwarding**

Review: Data Hazards & Forwarding

```
SUB   $s0,   $t0,   $t1      ;$s0 = $t0 - $t1
ADD   $t2,   $s0,   $t3      ;$t2 = $s0 + $t3
```

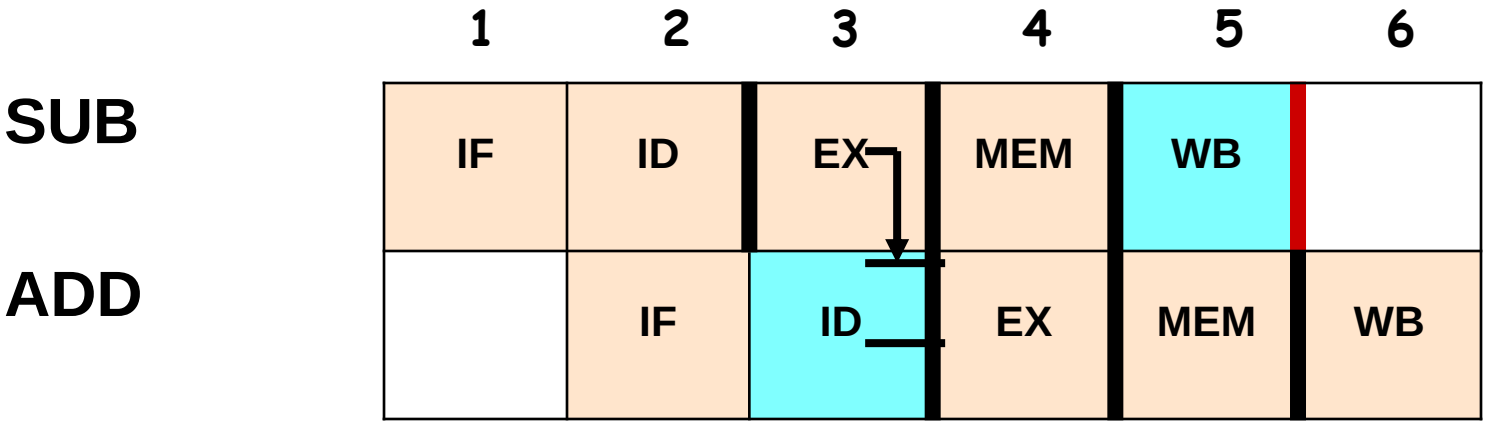


- **EX Hazard:** SUB result not written until its WB, ready at end of its EX, needed at start of ADD's EX
- **EX/MEM Forwarding:** forward $s0 from EX/MEM to ALU input in ADD EX stage (CC4)

Note: can occur in sequential instructions

Review: Data Hazards & Forwarding

```
SUB   $s0,   $t0,   $t1      ;$s0 = $t0 - $t1
ADD   $t2,   $s0,   $t3      ;$t2 = $s0 + $t3
```



EX Hazard Detection - EX/MEM Forwarding Conditions:

If ((EX/MEM.RegWrite = 1) & (EX/MEM.RegRD = ID/EX.RegRS))

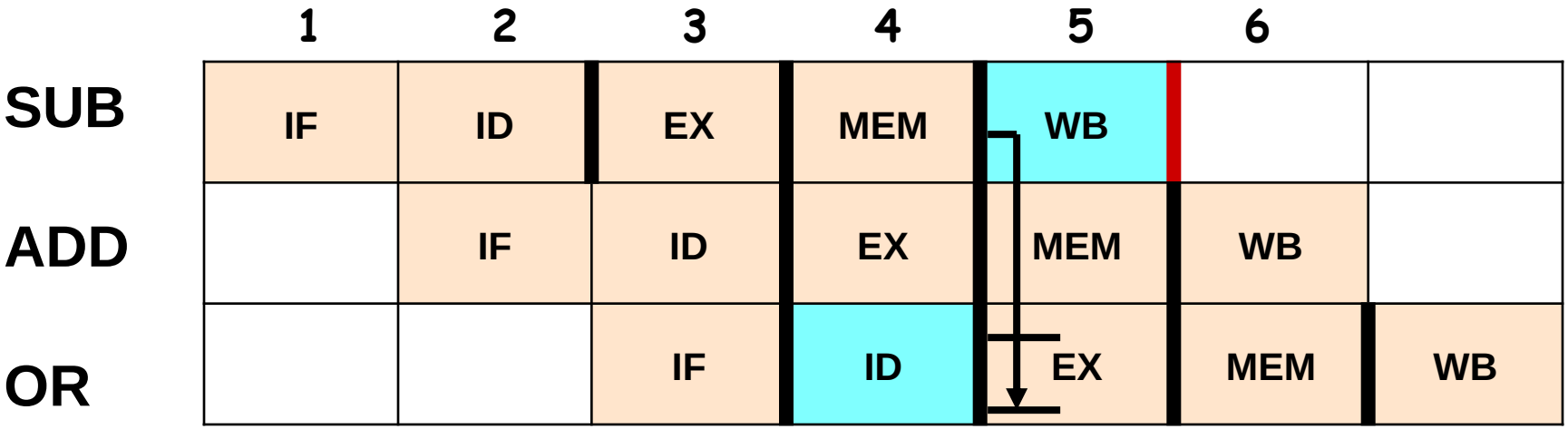
If ((EX/MEM.RegWrite = 1) & (EX/MEM.RegRD = ID/EX.RegRT))

Then forward EX/MEM result to EX stage

Note: In PH3, also check that EX/MEM.RegRD ≠ 0

Review: Data Hazards & Forwarding

```
SUB  $s0,  $t4,  $s3     ;$s0 = $t4 + $s3
ADD  $t2,  $s1,  $t1     ;$t2 = $s0 + $t1
OR   $s2,  $t3,  $s0     ;$s2 = $t3 OR $s0
```

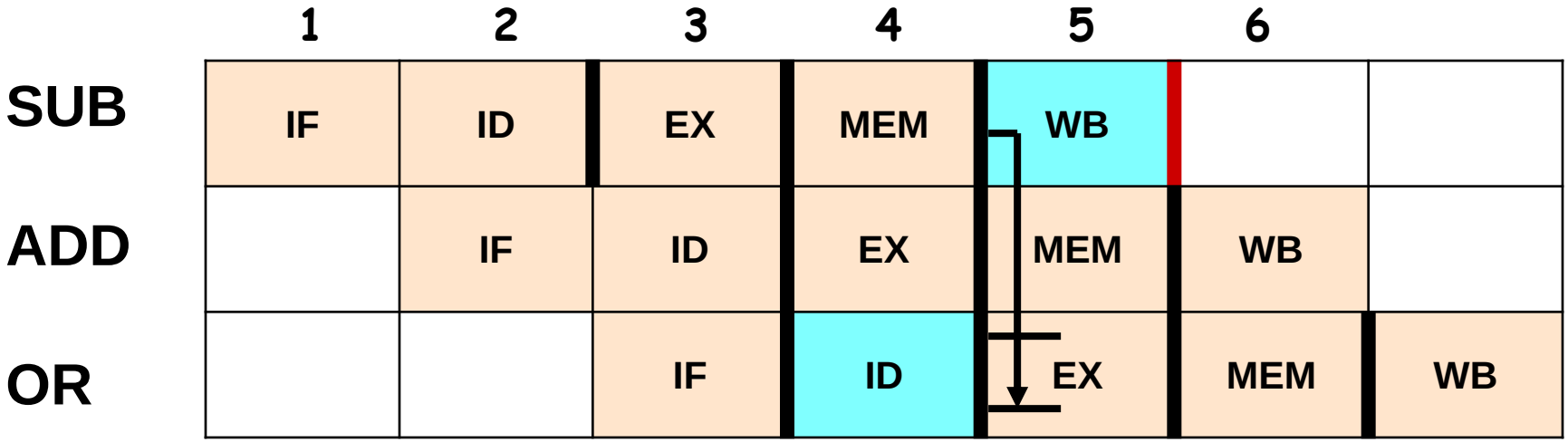


- **MEM Hazard:** SUB result not written until its WB, stored in MEM/WB, needed at start of OR's EX
- **MEM/WB Forwarding:** forward $s0 from MEM/WB to ALU input in OR EX stage (CC5)

Note: can occur in instructions I_n & I_{n+2}

Review: Data Hazards & Forwarding

```
SUB   $s0,   $t4,   $s3      ;$s0 = $t4 + $s3
ADD   $t2,   $s1,   $t1      ;$t2 = $s0 + $t1
OR    $s2,   $t3,   $s0      ;$s2 = $t3 OR $s0
```



MEM Hazard Detection - MEM/WB Forwarding Conditions:

If ((MEM/WB.RegWrite = 1) & (MEM/WB.RegRD = ID/EX.RegRS))

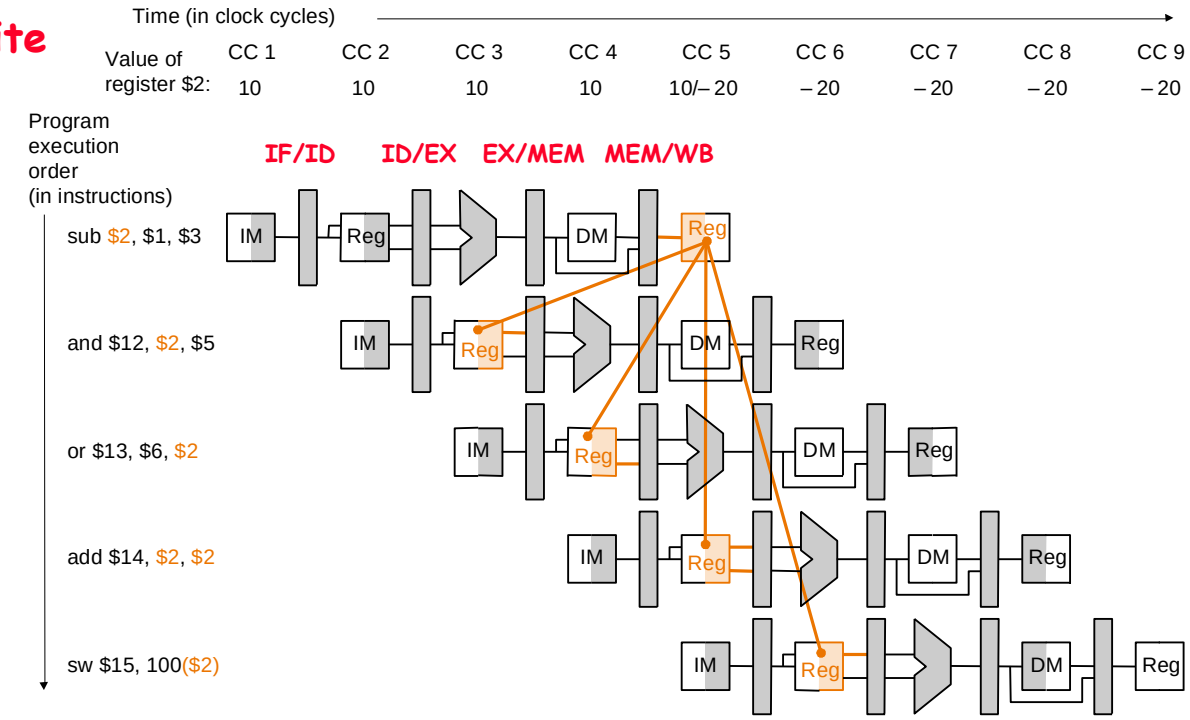
If ((EX/MEM.RegWrite = 1) & (EX/MEM.RegRD = ID/EX.RegRT))

Then forward MEM/WB result to EX stage

Note: In PH3, also check that MEM/WB.RegRD ≠ 0

Data Hazard Detection in MIPS

Read after Write



1a: EX/MEM.RegisterRd = ID/EX.RegisterRs
1b: EX/MEM.RegisterRd = ID/EX.RegisterRt } EX hazard

2a: MEM/WB.RegisterRd = ID/EX.RegisterRs
2b: MEM/WB.RegisterRd = ID/EX.RegisterRt } MEM hazard

Problem?

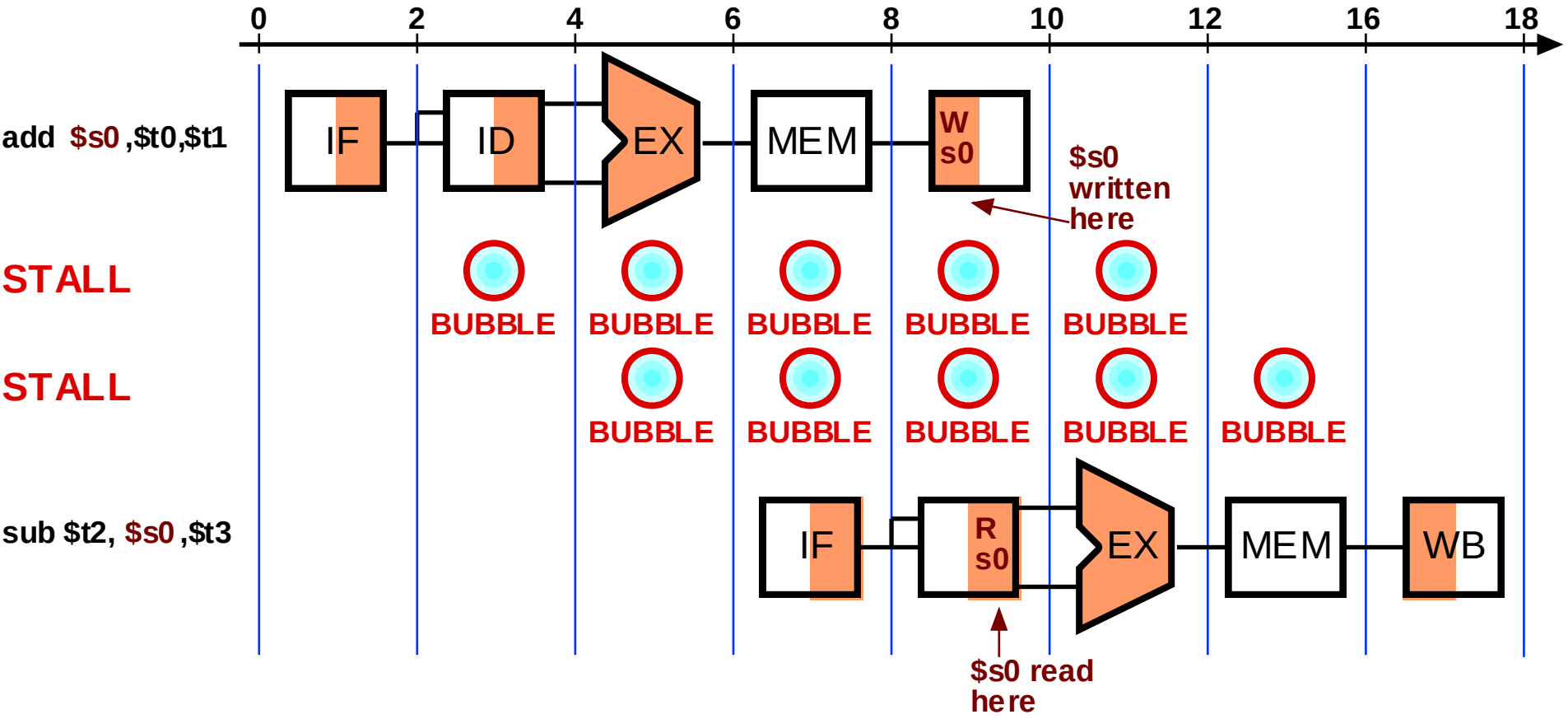
Some instructions do not write register.

EX/MEM.RegWrite must be asserted!

Data Hazards

- **Solutions for Data Hazards**
  - **Stalling**
  - **Forwarding:**
    - » **connect new value directly to next stage**
  - **Reordering**

Data Hazard - Stalling

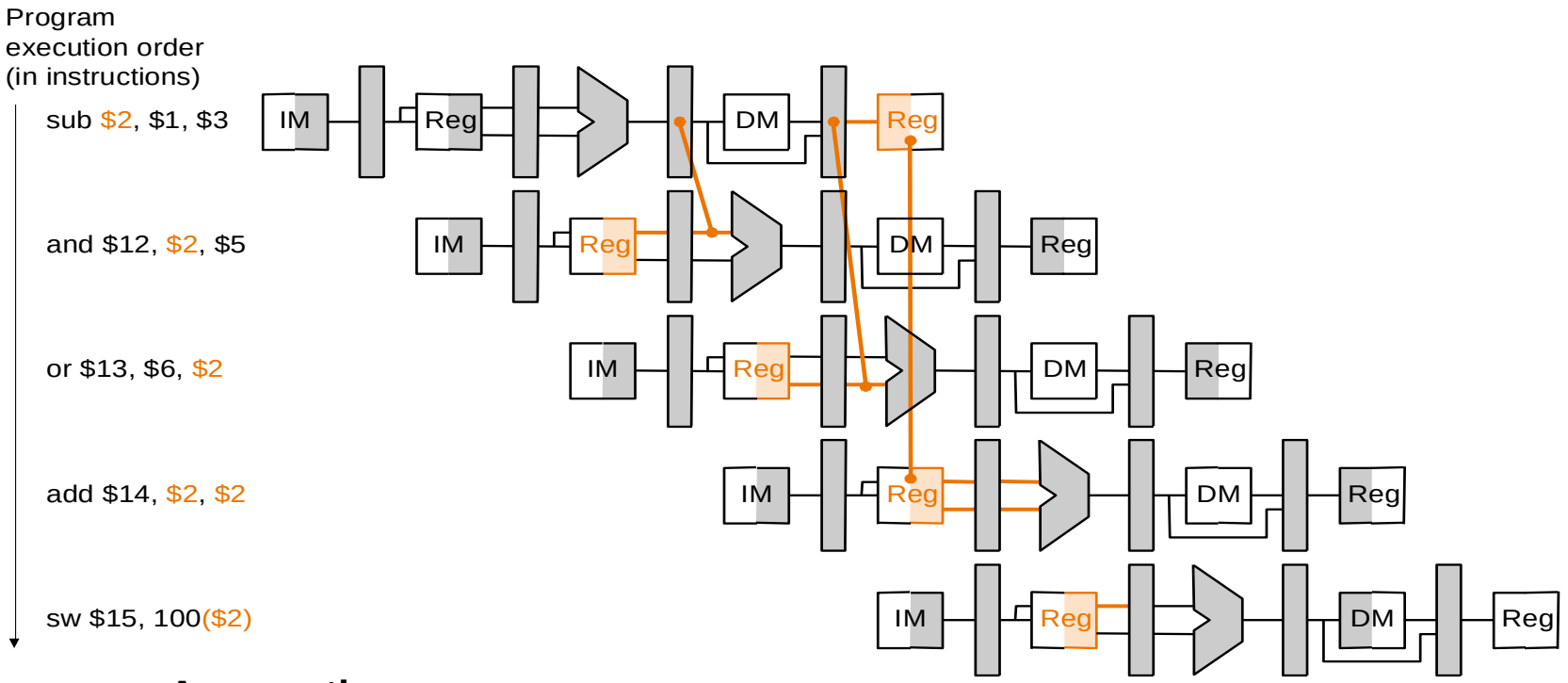


Data Hazard Solution: Forwarding

- **Key idea: connect data internally before it's stored**

Time (in clock cycles) →

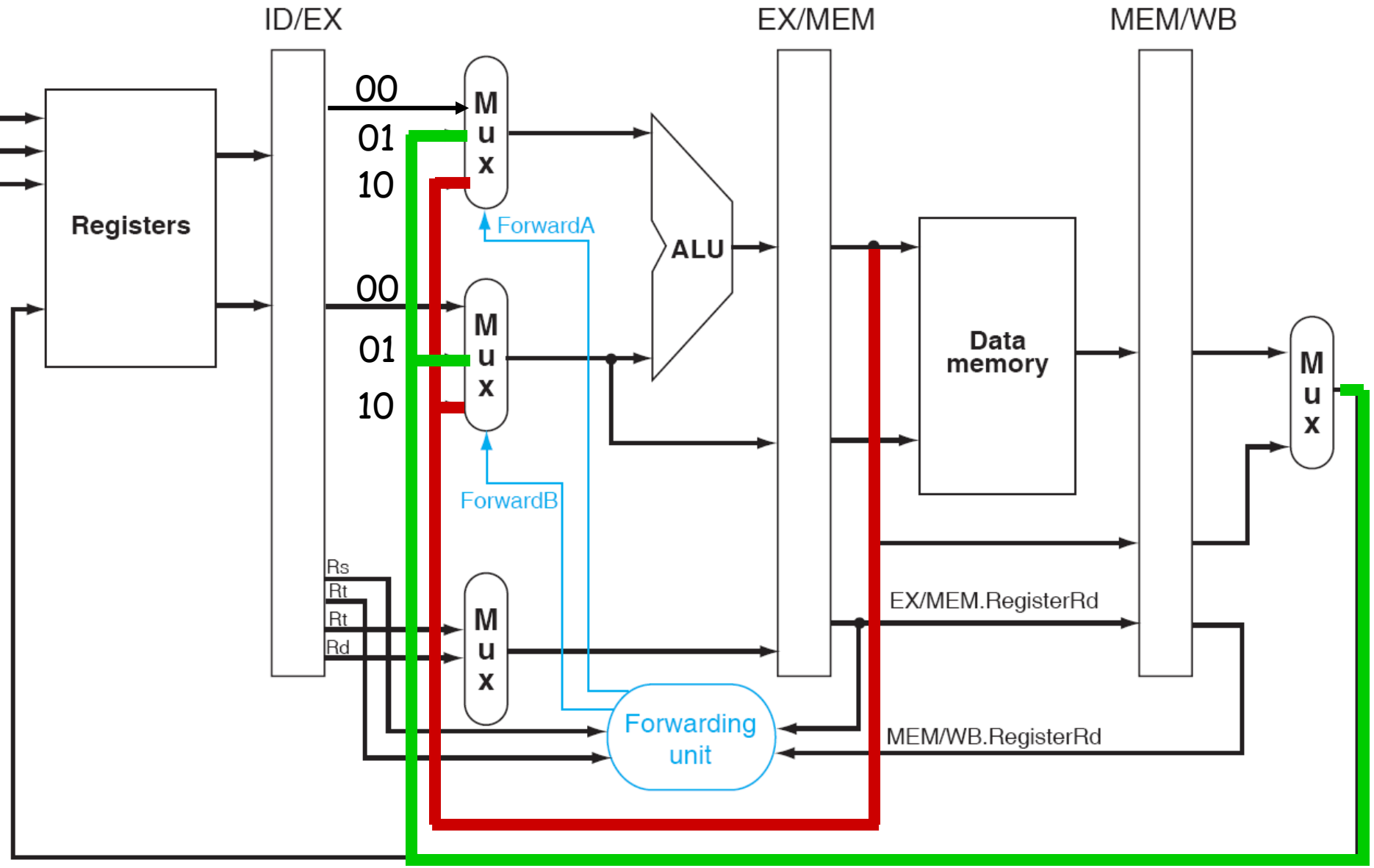
	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
Value of register $2 :	10	10	10	10	10/– 20	– 20	– 20	– 20	– 20
Value of EX/MEM :	X	X	X	– 20	X	X	X	X	X
Value of MEM/WB :	X	X	X	X	– 20	X	X	X	X



Assumption:

- **The register file forwards values that are read and written during the same cycle.**

Forwarding



Add hardware to feed back ALU and MEM results to both ALU inputs

Controlling Forwarding

- **Need to test when register numbers match in rs, rt, and rd fields stored in pipeline registers**
- **"EX" hazard:**
  - **EX/MEM - test whether instruction writes register file and examine rd register**
  - **ID/EX - test whether instruction reads rs or rt register and matches rd register in EX/MEM**
- **"MEM" hazard:**
  - **MEM/WB - test whether instruction writes register file and examine rd (rt) register**
  - **ID/EX - test whether instruction reads rs or rt register and matches rd (rt) register in EX/MEM**

Forwarding Unit Detail - EX Hazard

if (EX/MEM.RegWrite)

and (EX/MEM.RegisterRd ≠ 0)

and (EX/MEM.RegisterRd = ID/EX.RegisterRs))

ForwardA = 10

if (EX/MEM.RegWrite)

and (EX/MEM.RegisterRd ≠ 0)

and (EX/MEM.RegisterRd = ID/EX.RegisterRt))

ForwardB = 10

Forwarding Unit Detail - MEM Hazard

if (MEM/WB.RegWrite)

and (MEM/WB.RegisterRd ≠ 0)

and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01

if (MEM/WB.RegWrite)

and (MEM/WB.RegisterRd ≠ 0)

and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01

Data Hazards and Stalls

- So far, we've only addressed "potential" data hazards, where the forwarding unit was able to detect and resolve them without affecting the performance of the pipeline.

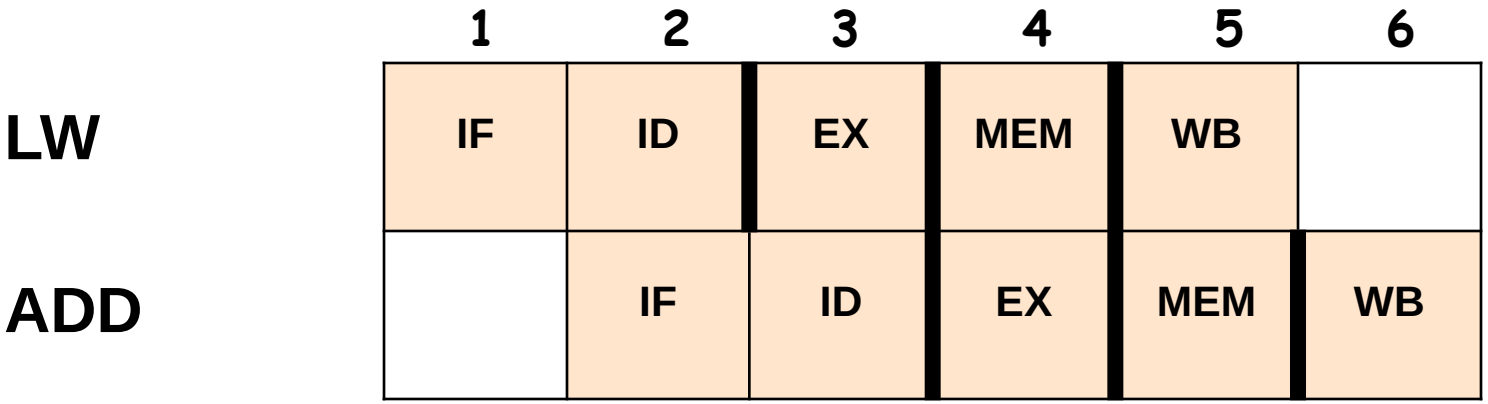
- There are also "unavoidable" data hazards, which the forwarding unit cannot resolve, and whose resolution does affect pipeline performance.

- We thus add a (unavoidable) hazard detection unit, which detects them and introduces stalls to resolve them.

Data Hazards & Stalls

- **Identify the true data hazard in this sequence:**

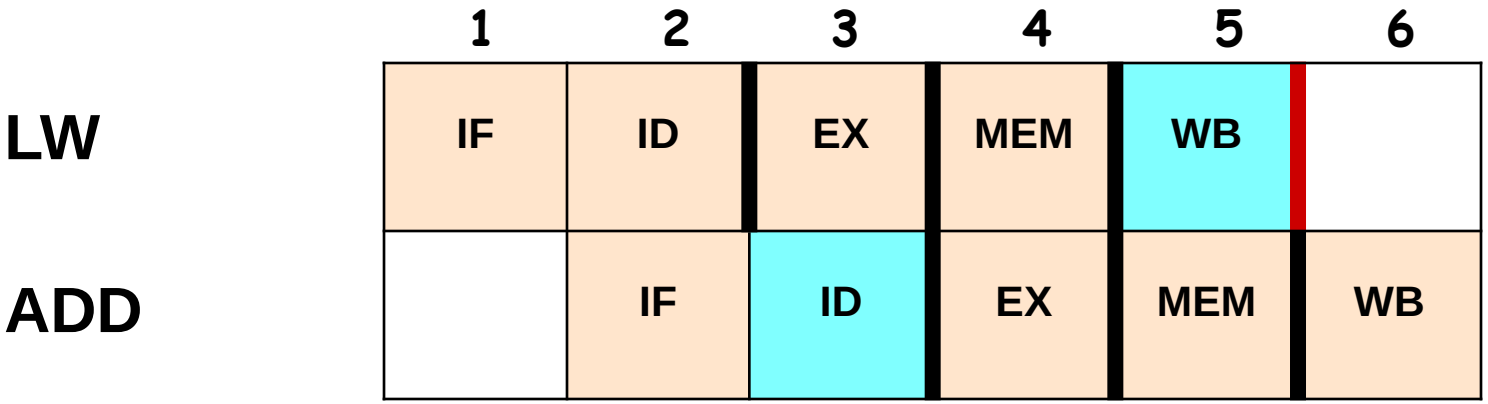
```
LW    $s0,   100($t0)      ;$s0 = memory value
ADD   $t2,   $s0,   $t3     ;$t2 = $s0 + $t3
```



Data Hazards & Stalls

- **Identify the true data hazard in this sequence:**

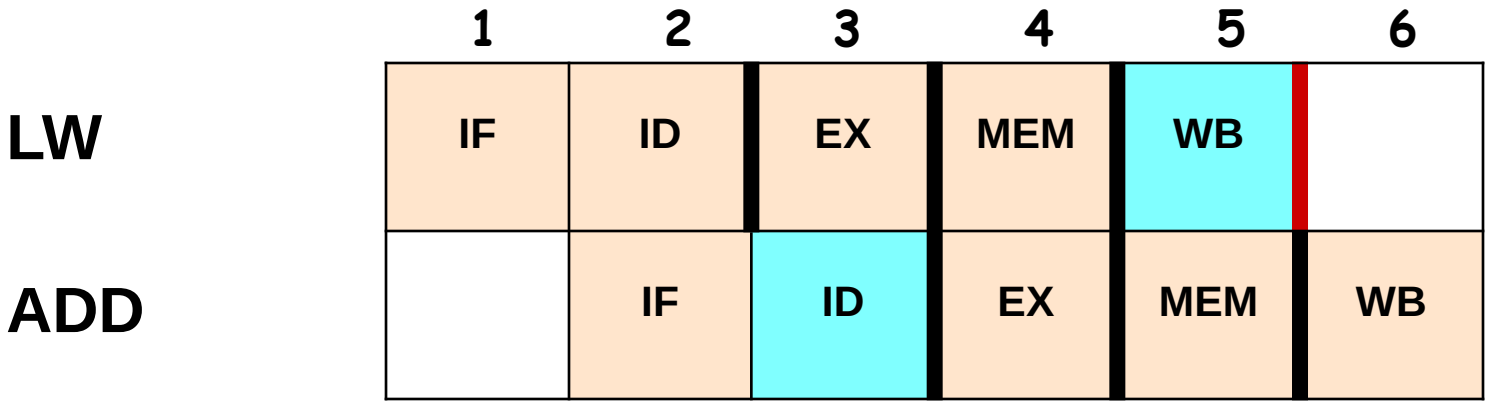
```
LW    $s0,  100($t0)      ;$s0 = memory value
ADD   $t2,  $s0,  $t3     ;$t2 = $s0 + $t3
```



- **LW doesn't write $s0 to Reg File until the end of CC5, but ADD reads $s0 from Reg File in CC3**

Data Hazards & Stalls

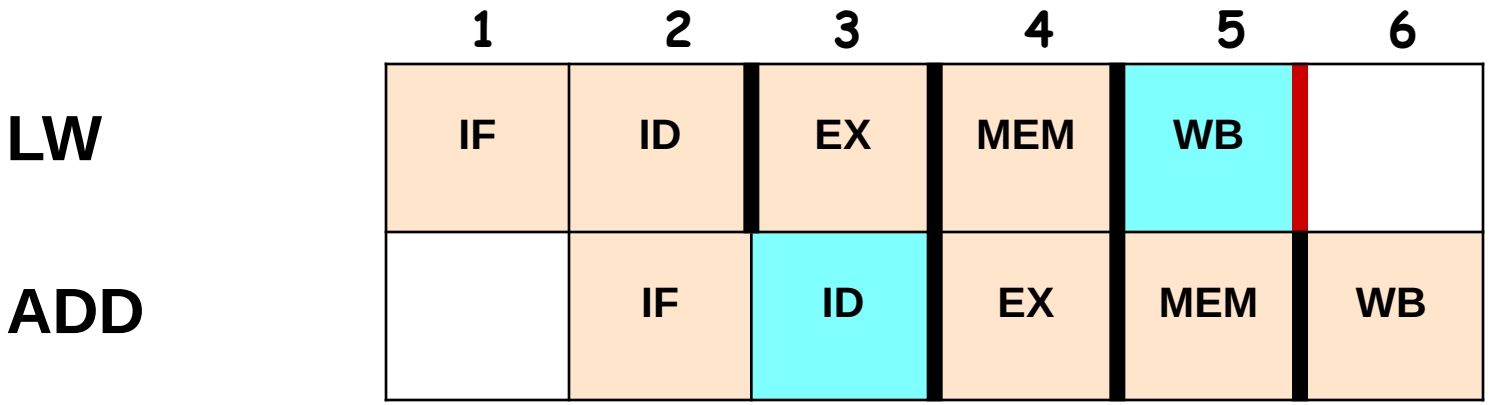
```
LW    $s0,  100($t0)      ;$s0 = memory value
ADD   $t2,  $s0,  $t3     ;$t2 = $s0 + $t3
```



- **EX/MEM forwarding won't work, because the data isn't loaded from memory until CC4 (so it's not in EX/MEM register)**

Data Hazards & Stalls

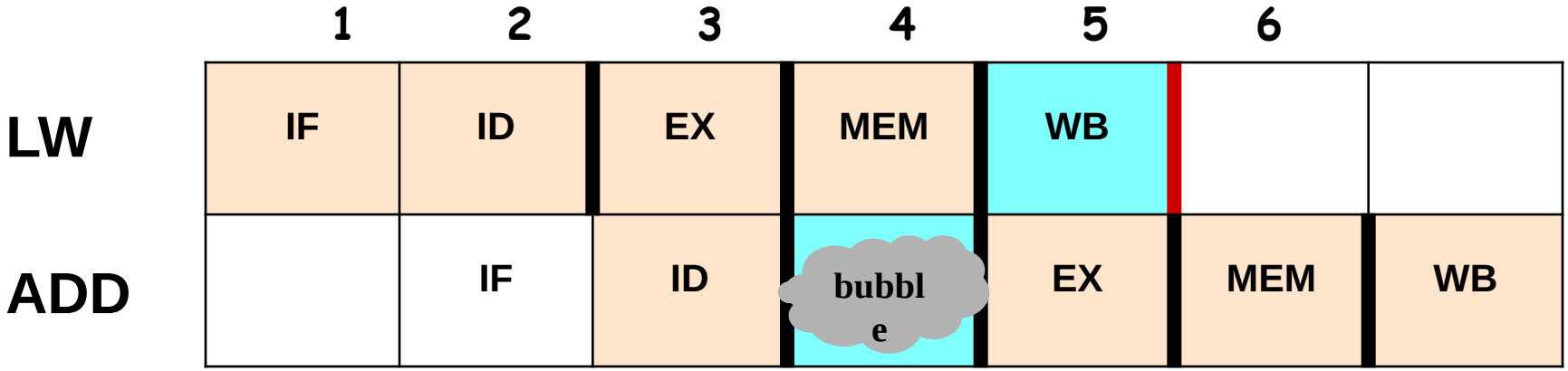
```
LW    $s0,  100($t0)      ;$s0 = memory value
ADD   $t2,  $s0,  $t3     ;$t2 = $s0 + $t3
```



- **MEM/WB forwarding won't work either, because ADD executes in CC4**

Data Hazards & Stalls: implementation

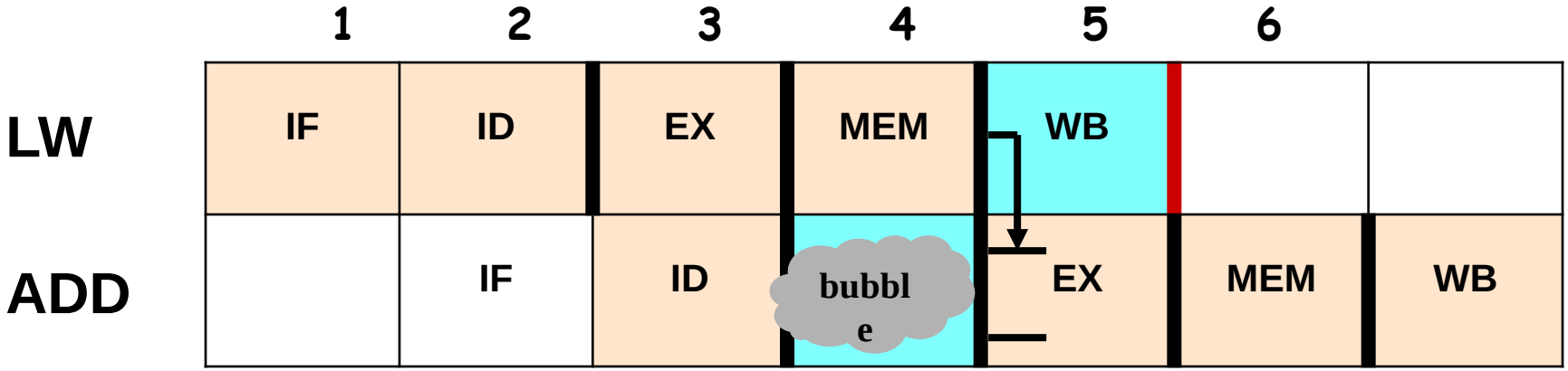
```
LW    $s0,  100($t0)      ;$s0 = memory value
ADD   $t2,  $s0,  $t3     ;$t2 = $s0 + $t3
```



- We must handle this hazard by "stalling" the pipeline for 1 Clock Cycle (bubble)

Data Hazards & Stalls: implementation

```
LW    $s0,  100($t0)      ;$s0 = memory value
ADD   $t2,  $s0,  $t3     ;$t2 = $s0 + $t3
```

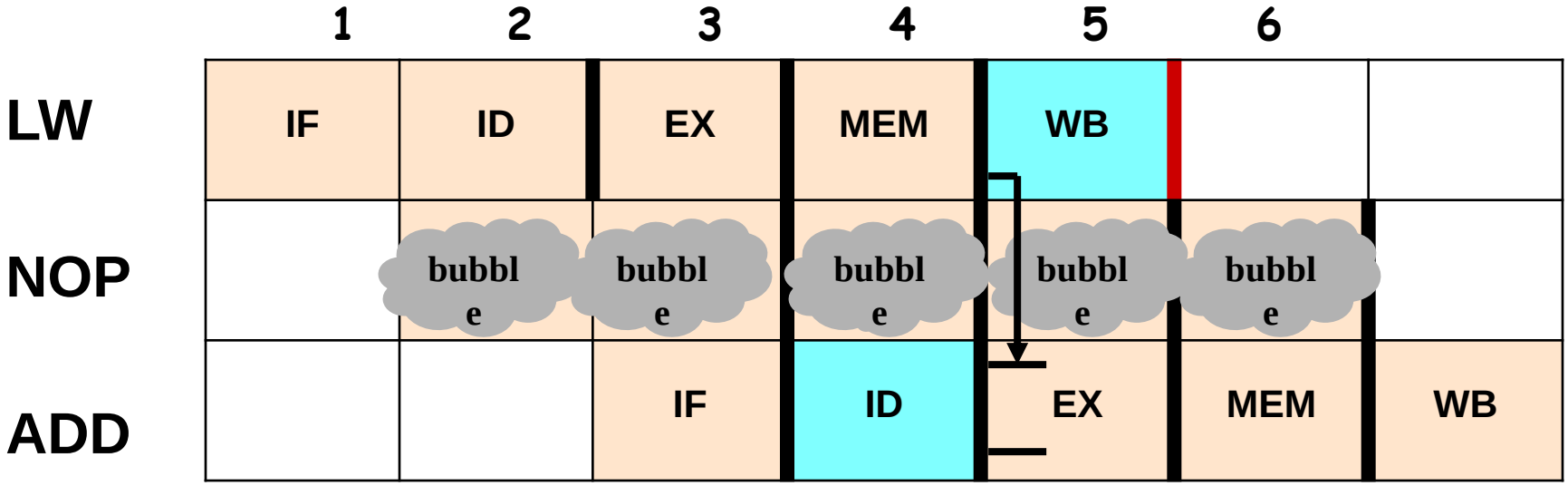


- **We can then use MEM/WB forwarding, but of course there is still a performance loss**

Data Hazards & Stalls: implementation

- **Stall Implementation #1:** Compiler detects hazard and inserts a NOP (no reg changes (SLL $0, $0, 0))

```
LW    $s0,   100($t0)      ;$s0 = memory value
NOP      ;dummy instruction
ADD   $t2,   $s0,   $t3      ;$t2 = $s0 + $t3
```



- **Problem: we have to rely on the compiler**

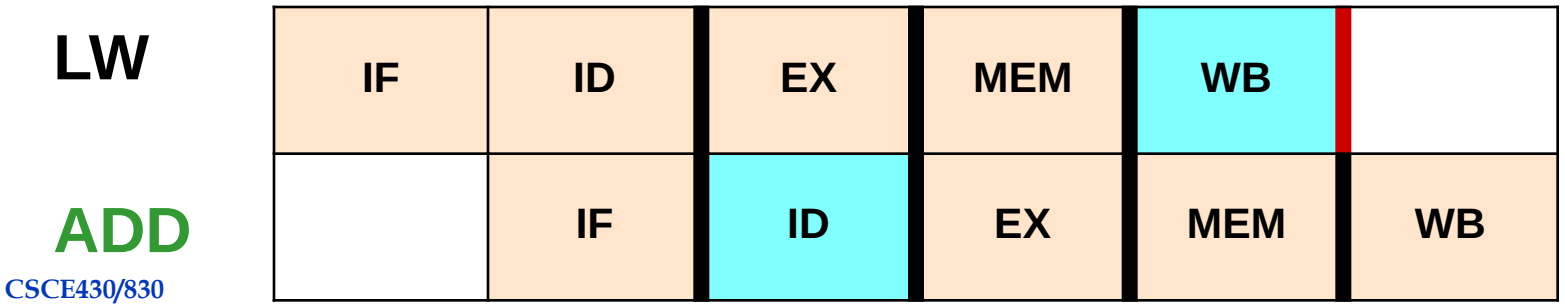
Data Hazards & Stalls: implementation

- **Stall Implementation #2:** Add a "hazard detection unit" to stall current instruction for 1 CC if:

- **ID-Stage Hazard Detection and Stall Condition:**

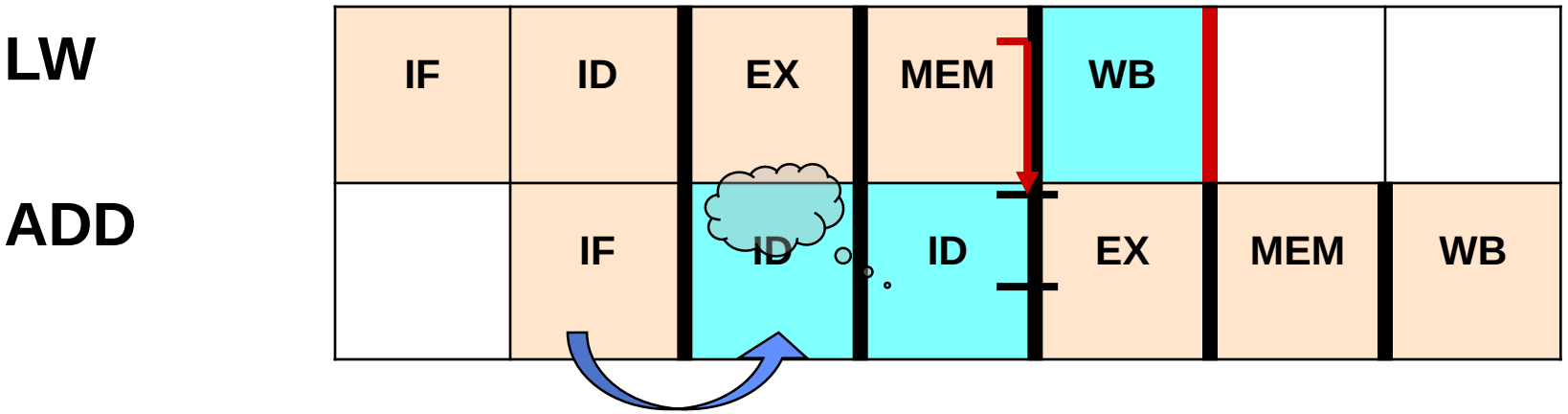
```
If ((ID/EX.MemRead = 1) &          ;only a LW reads mem
((ID/EX.RegRT = IF/ID.RegRS) ||    ;RS will read load dest (RT)
(ID/EX.RegRT = IF/ID.RegRT)))      ;RT will read load dest
```

```
LW    $s0,   100($t0)      ;$s0 = memory value
ADD   $t2,   $s0,   $t3    ;$t2 = $s0 + $t3
```



Data Hazards & Stalls: implementation

- **The effect of this stall will be to repeat the ID Stage of the current instruction. Then we do the MEM/WB forwarding on the next Clock Cycle**



- **We do this by preserving the current values in IF/ID for use on the next Clock Cycle**

Data Hazards: A Classic Example

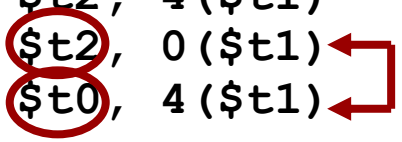
- **Identify the data dependencies in the following code. Which of them can be resolved through forwarding?**

```
SUB   $2,    $1,   $3
OR    $12,  $2,   $5
SW    $13, 100($2)
ADD   $14,  $2,   $2
LW    $15, 100($2)
ADD   $4,   $7,   $15
```

Data Hazards - Reordering Instructions

- **Assuming we have data forwarding, what are the hazards in this code?**

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t2, 0($t1)
sw $t0, 4($t1)
```



- **Reorder instructions to remove hazard:**

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t0, 4($t1)
sw $t2, 0($t1)
```

Data Hazard Summary

- **Three types of data hazards**
  - **RAW (MIPS)**
  - **WAW (not in MIPS)**
  - **WAR (not in MIPS)**
- **Solution to RAW in MIPS**
  - **Stall**
  - **Forwarding**
    - » **Detection & Control**
      - **EX hazard**
      - **MEM hazard**
    - » **A stall is needed if read a register after a load instruction that writes the same register.**
  - **Reordering**

Pipelining Outline
Next class

- **Introduction**
  - Defining Pipelining
  - Pipelining Instructions
- **Hazards**
  - Structural hazards
  - Data Hazards
  - Control Hazards ⇦
- **Performance**
- **Controller implementation**

Pipeline Hazards

- **Where one instruction cannot immediately follow another**
- **Types of hazards**
  - **Structural hazards - attempt to use same resource twice**
  - **Control hazards - attempt to make decision before condition is evaluated**
  - **Data hazards - attempt to use data before it is ready**
- **Can always resolve hazards by waiting**

Control Hazards

A *control hazard* is when we need to find the destination of a branch, and can't fetch any new instructions until we know that destination.

A branch is either

- **Taken: PC <= PC + 4 + Immediate**
- **Not Taken: PC <= PC + 4**

Control Hazards

Control Hazard on Branches
Three Stage Stall

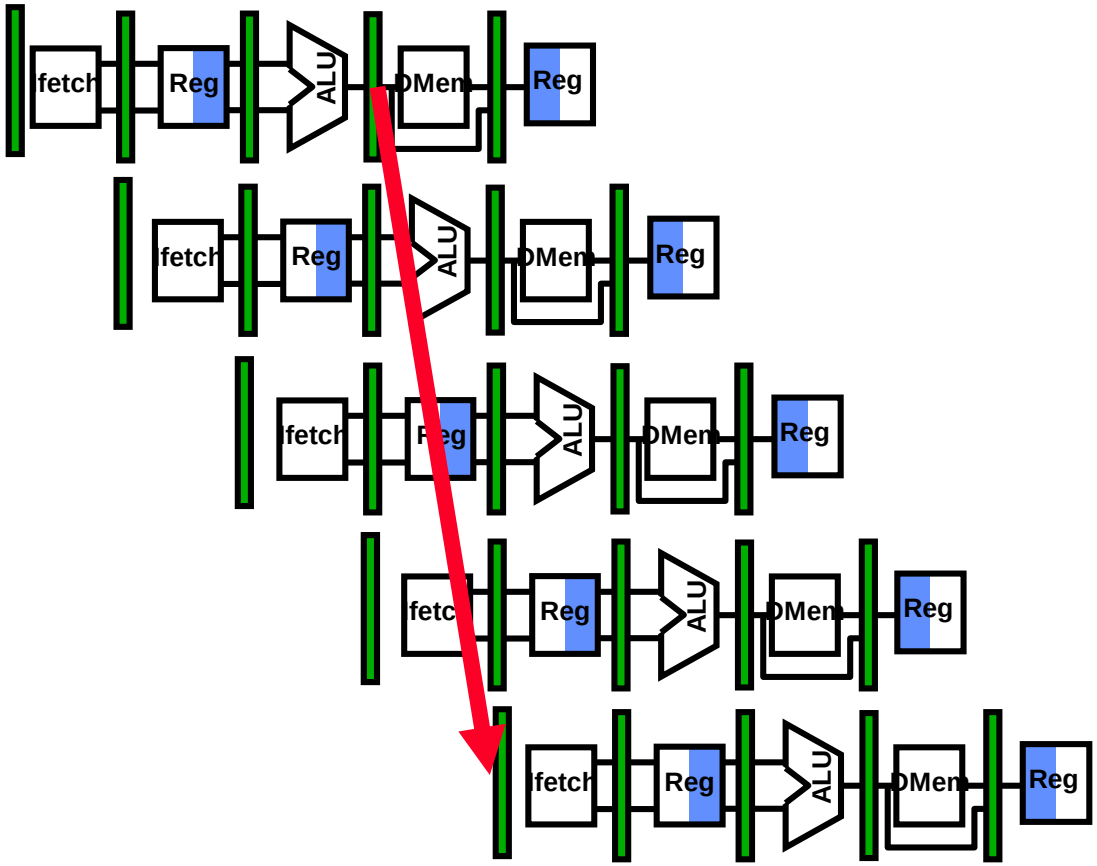
```
10: beq r1,r3,36

14: and r2,r3,r5

18: or  r6,r1,r7

22: add r8,r1,r9

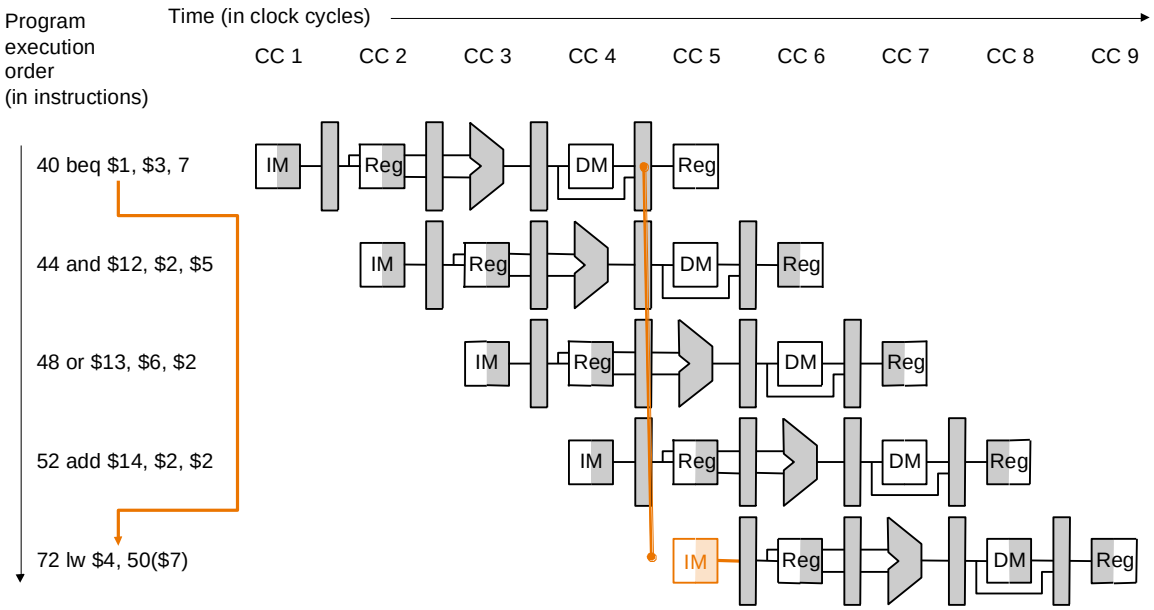
36: xor r10,r1,r11
```



The penalty when branch take is 3 cycles!

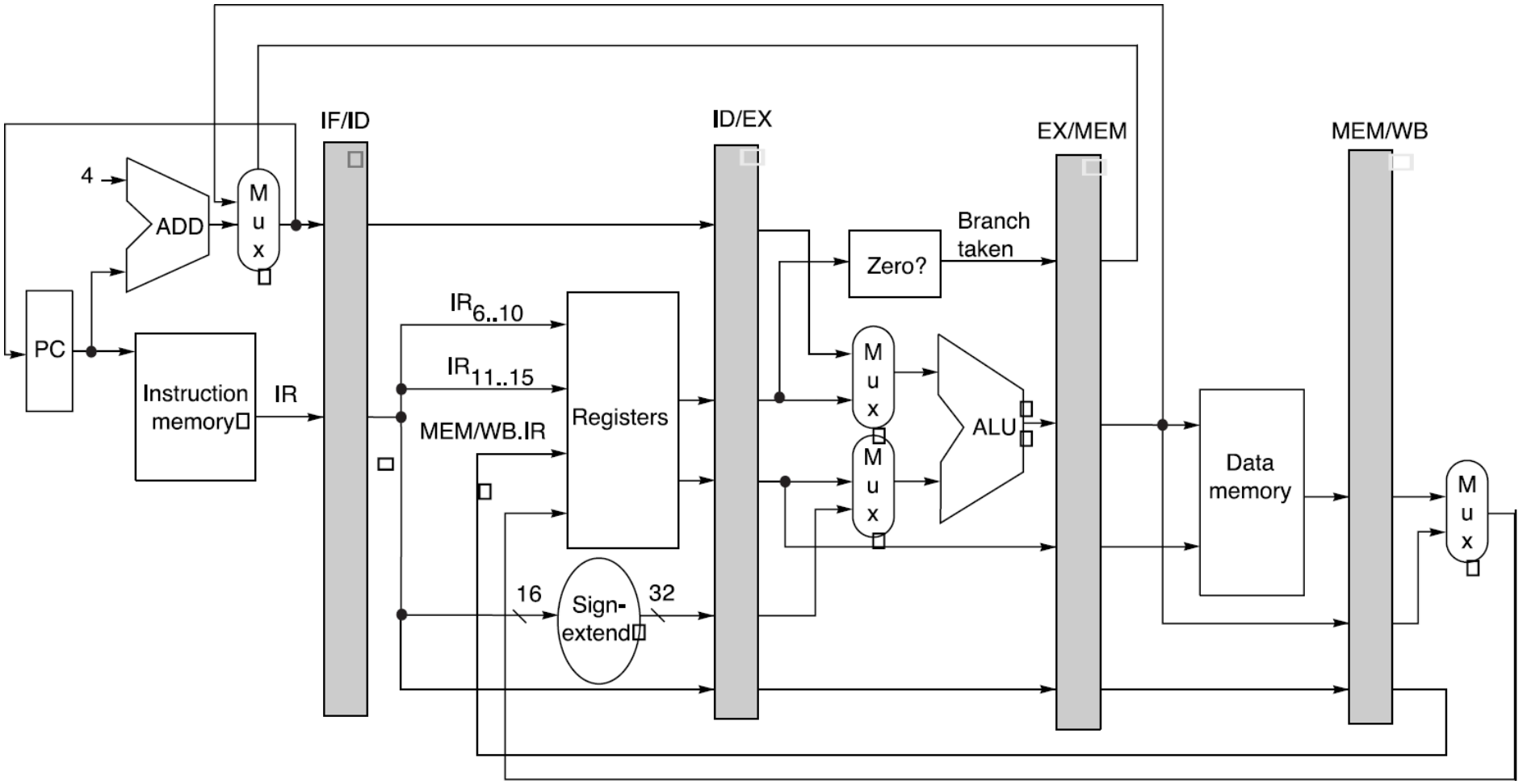
Branch Hazards

- **Just stalling for each branch is not practical**
- **Common assumption: branch not taken**
- **When assumption fails: flush three instructions**



(Fig. 6.37)

Basic Pipelined Processor



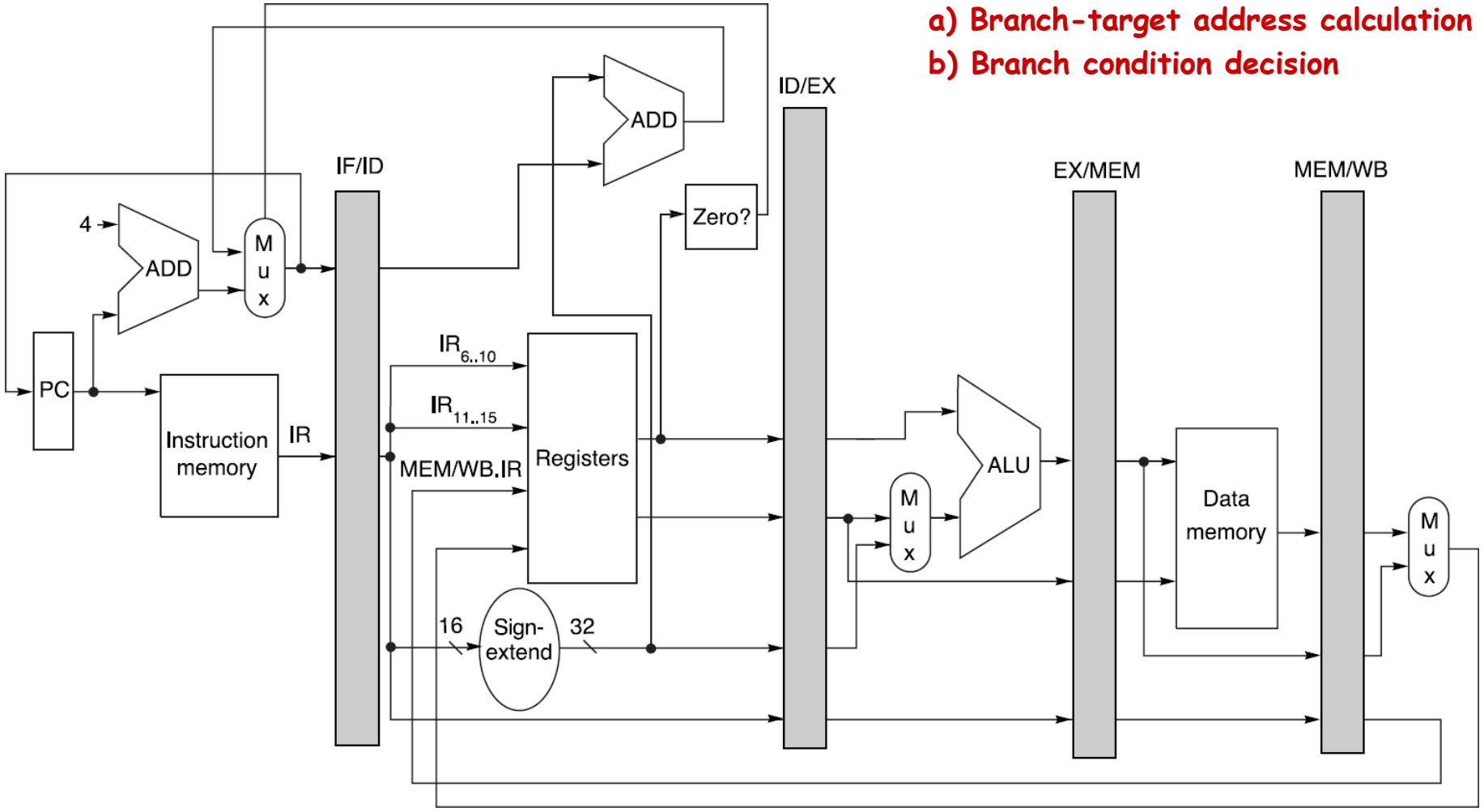
In our original Design, branches have a penalty of 3 cycles

Reducing Branch Delay

Move following to ID stage

a) Branch-target address calculation

b) Branch condition decision



Reduced penalty (1 cycle) when branch take!

Reducing Branch Delay

- **Key idea: move branch logic to ID stage of pipeline**
  - **New adder calculates branch target**
    `(PC + 4 + extend(IMM))`
  - **New hardware tests `rs == rt` after register read**
- **Reduced penalty (1 cycle) when branch take**

Control Hazard Solutions

- **Stall**
  - stop loading instructions until result is available
- **Predict**
  - assume an outcome and continue fetching (undo if prediction is wrong)
  - lose cycles only on *mis-prediction*
- **Delayed branch**
  - specify in architecture that the instruction immediately following branch is always executed

Branch Behavior in Programs

- **Based on SPEC benchmarks on DLX**
  - **Branches occur with a frequency of 14% to 16% in integer programs and 3% to 12% in floating point programs.**
  - **About 75% of the branches are forward branches**
  - **60% of forward branches are taken**
  - **80% of backward branches are taken**
  - **67% of all branches are taken**
- **Why are branches (especially backward branches) more likely to be taken than not taken?**

Static Branch Prediction

For every branch encountered during execution predict whether the branch will be taken or not taken.

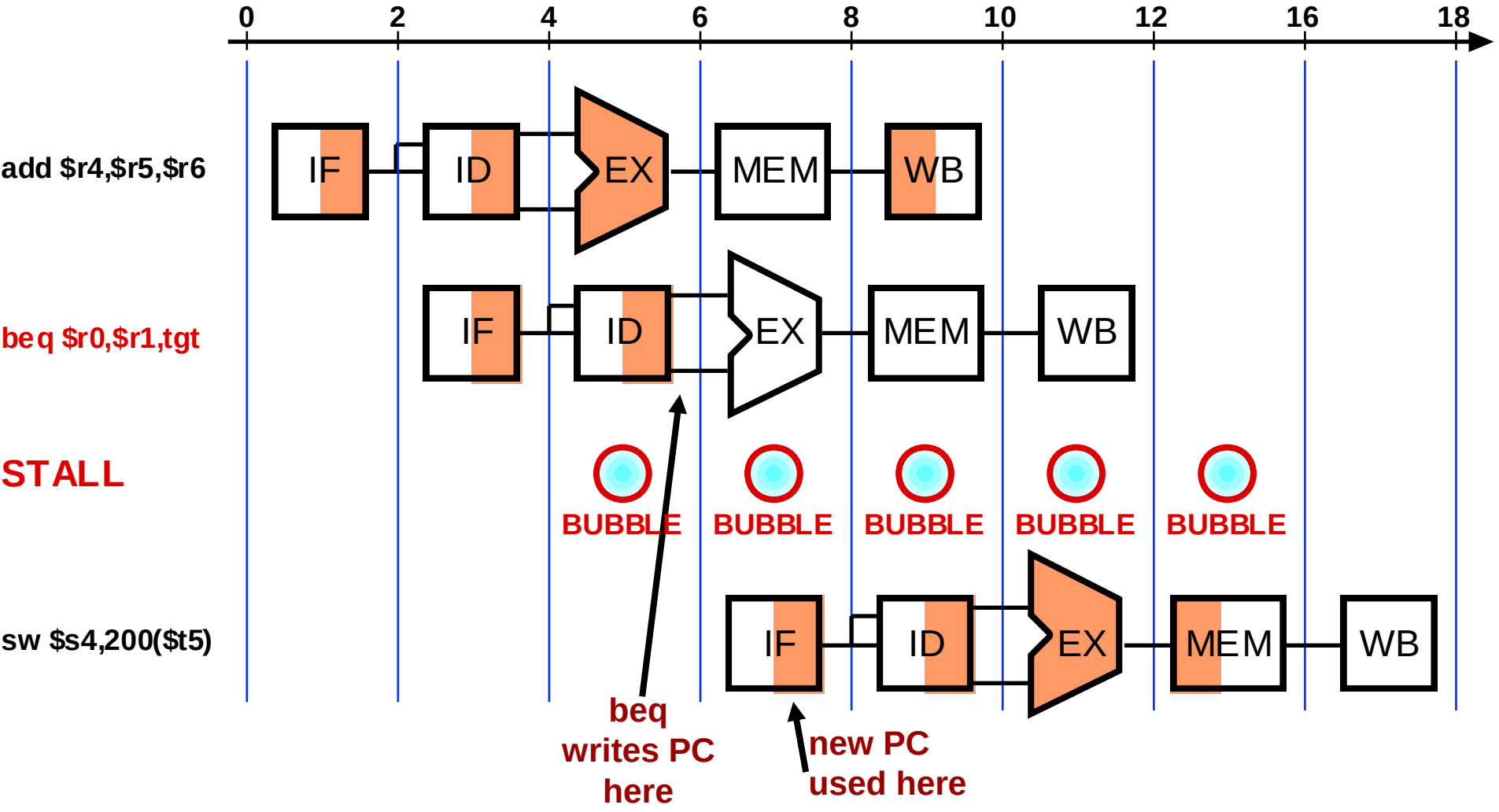
Predicting branch not taken:

1. **Speculatively fetch and execute in-line instructions following the branch**
2. **If prediction incorrect flush pipeline of speculated instructions**
   - **Convert these instructions to NOPs by clearing pipeline registers**
   - **These have not updated memory or registers at time of flush**

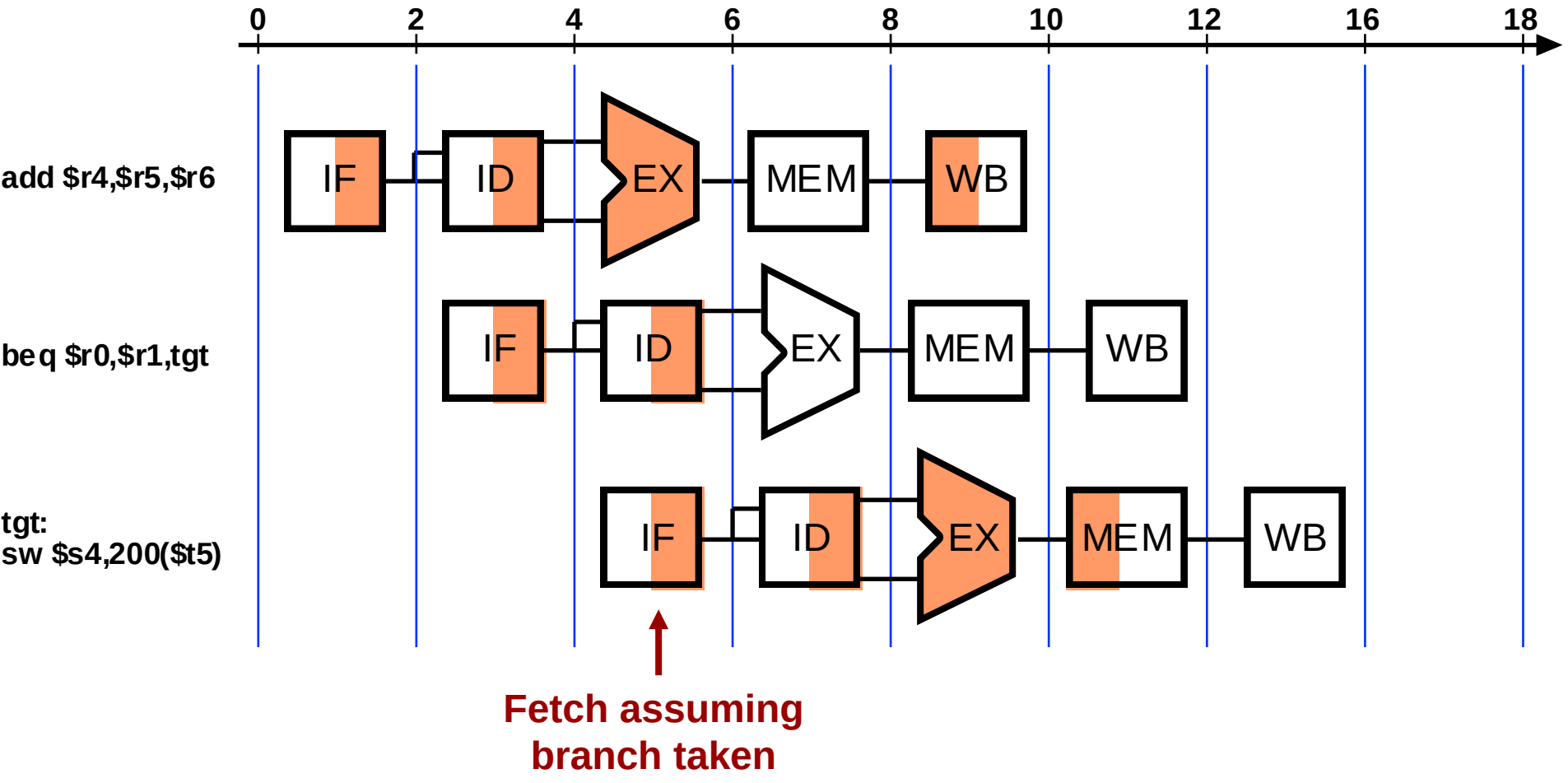
Predicting branch taken:

1. **Speculatively fetch and execute instructions at the branch target address**
2. **Useful only if target address known earlier than branch outcome**
   - **May require stall cycles till target address known**
   - **Flush pipeline if prediction is incorrect**
   - **Must ensure that flushed instructions do not update memory/registers**

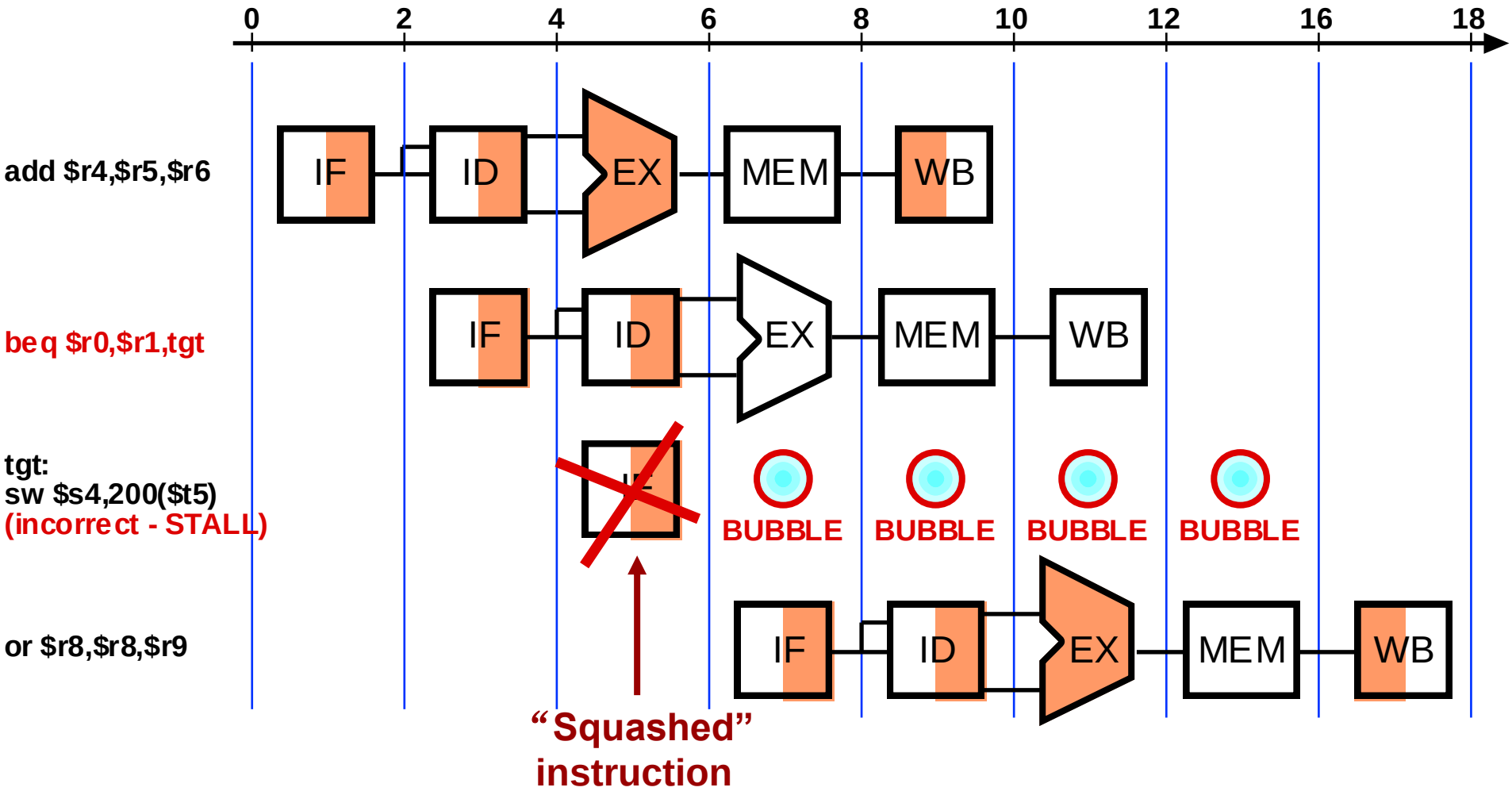
Control Hazard - Stall



Control Hazard - Correct Prediction



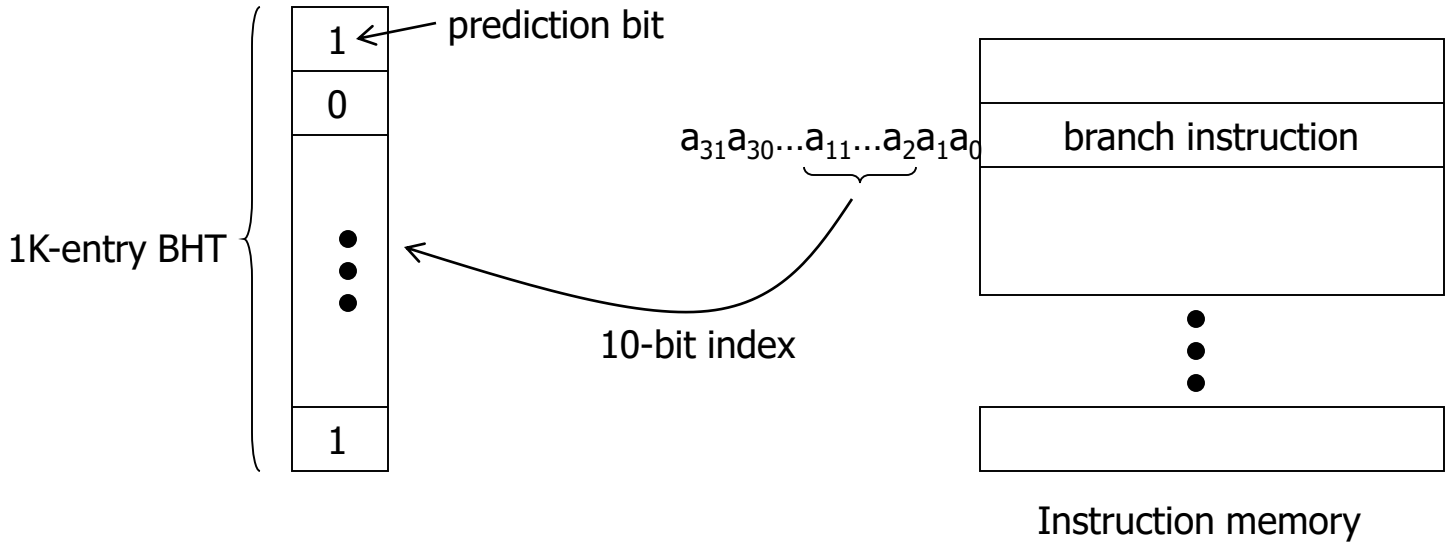
Control Hazard - Incorrect Prediction



1-Bit Branch Prediction

- **Branch History Table (BHT): Lower bits of PC address index table of 1-bit values**
  - **Says whether or not the branch was taken last time**
  - **No address check (saves HW, but may not be the right branch)**
  - **If prediction is wrong, invert prediction bit**

1 = branch was last taken
0 = branch was last not taken



Hypothesis: branch will do the same again.

1-Bit Branch Prediction

- **Example:**

  Consider a loop branch that is taken 9 times in a row and then not taken once. What is the prediction accuracy of the 1-bit predictor for this branch assuming only this branch ever changes its corresponding prediction bit?

1-Bit Branch Prediction

- **Example:**

  Consider a loop branch that is taken 9 times in a row and then not taken once. What is the prediction accuracy of the 1-bit predictor for this branch assuming only this branch ever changes its corresponding prediction bit?

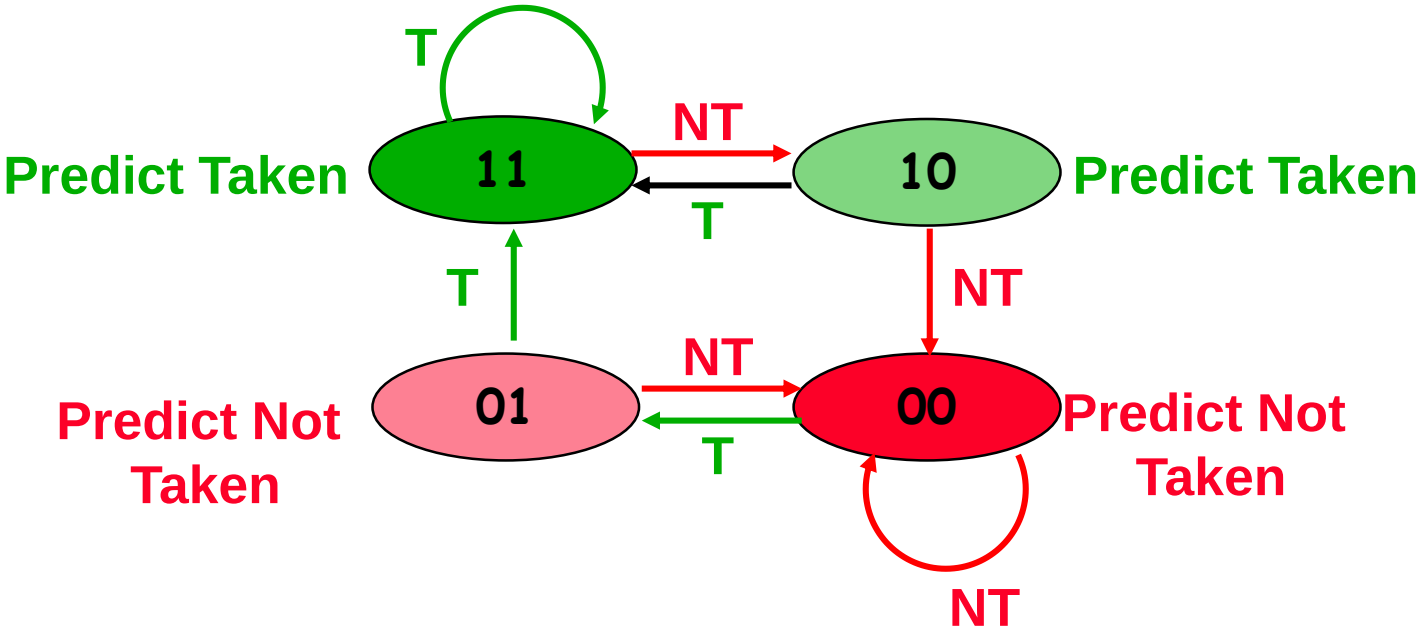
  - **Answer: *80%.* Because there are two mispredictions – one on the first iteration and one on the last iteration. *Is this good enough and Why?***

2-Bit Branch Prediction
(Jim Smith, 1981)

- **Solution: a 2-bit scheme where prediction is changed only if mispredicted *twice***

  Red: stop, not taken

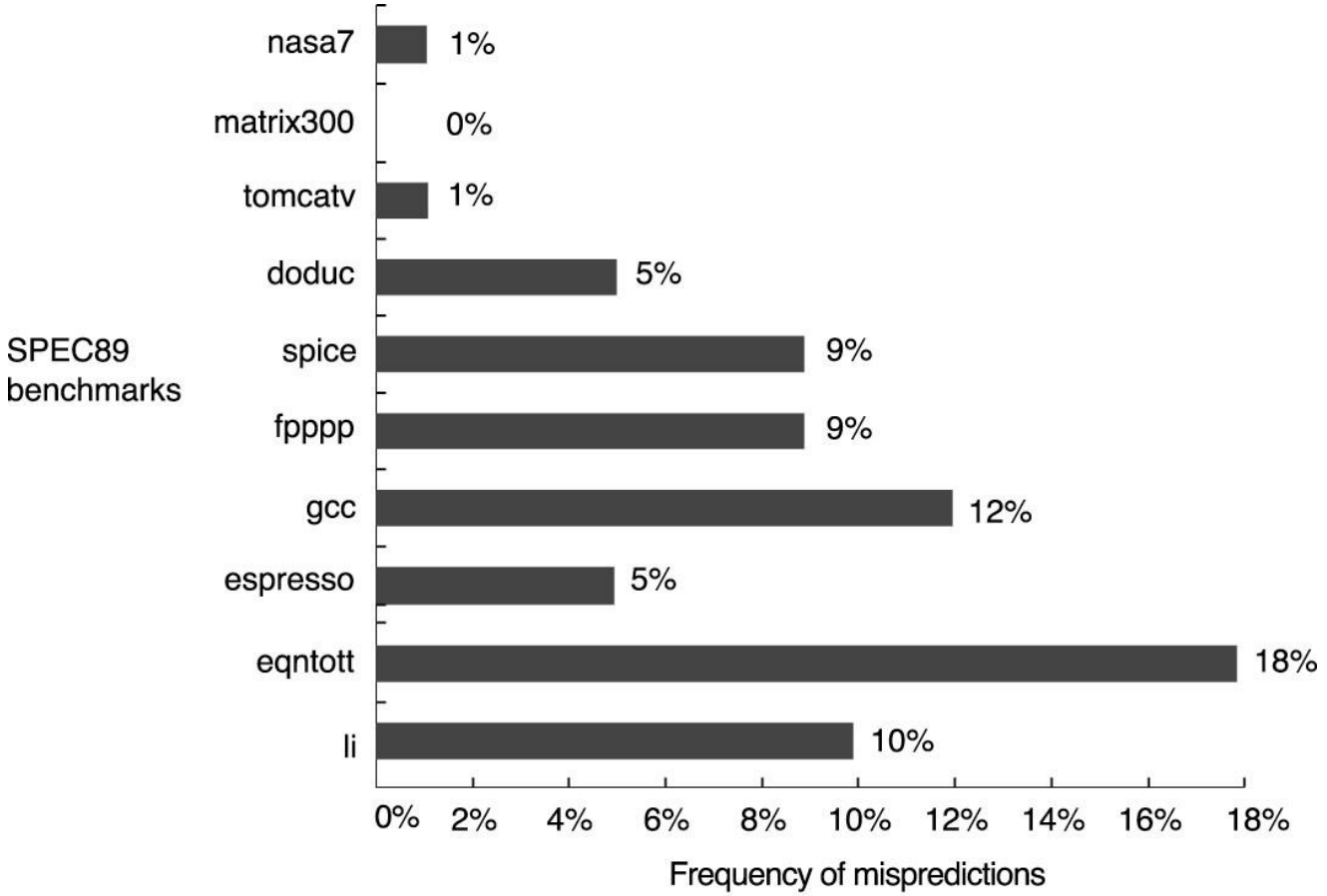
  Green: go, taken



n-bit Saturating Counter

- **Values: 0 ~ 2^n-1**
- **When the counter is greater than or equal to one-half of its maximum value, the branch is predicted as taken. Otherwise, not taken.**
- **Studies have shown that the 2-bit predictors do almost as well, and thus most systems rely on 2-bit branch predictors.**

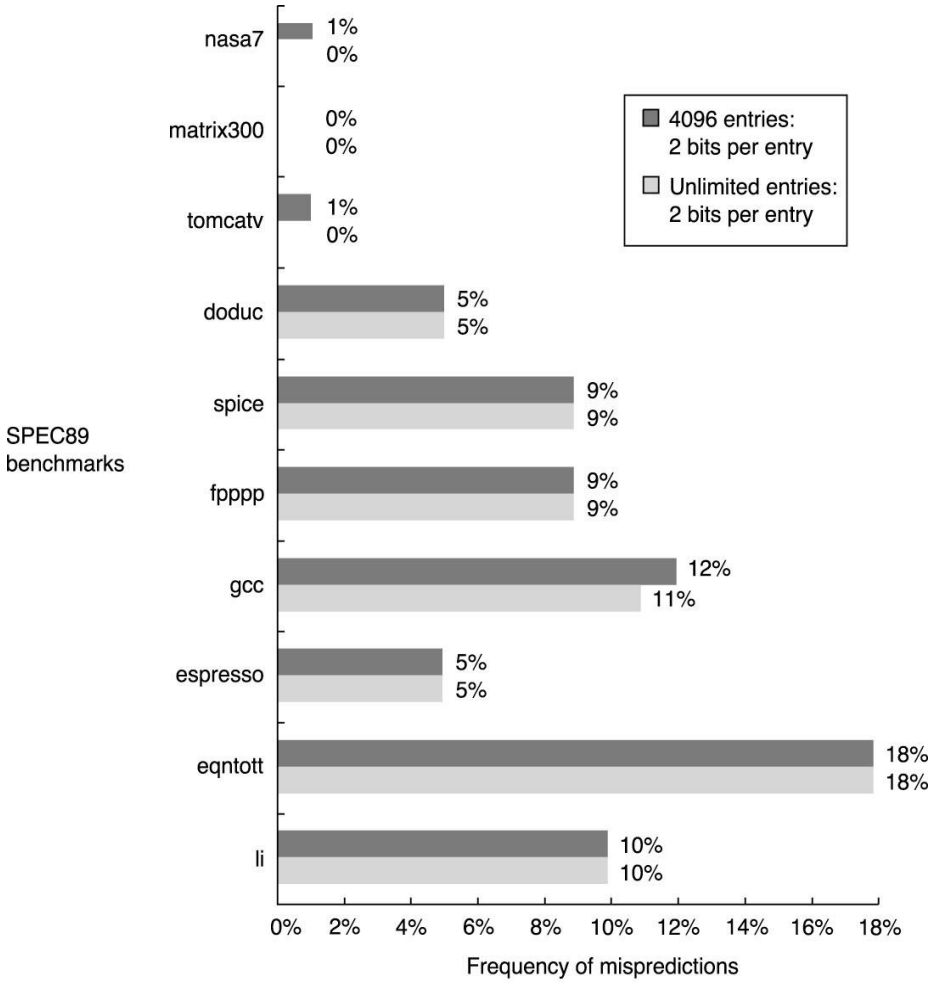
2-bit Predictor Statistics



Prediction accuracy of 4K-entry 2-bit prediction buffer on SPEC89 benchmarks: accuracy is lower for integer programs (gcc, espresso, eqntott, li) than for FP

© 2003 Elsevier Science (USA). All rights reserved.

2-bit Predictor Statistics



Prediction accuracy of 4K-entry 2-bit prediction buffer vs. "infinite" 2-bit buffer: increasing buffer size from 4K does not significantly improve performance

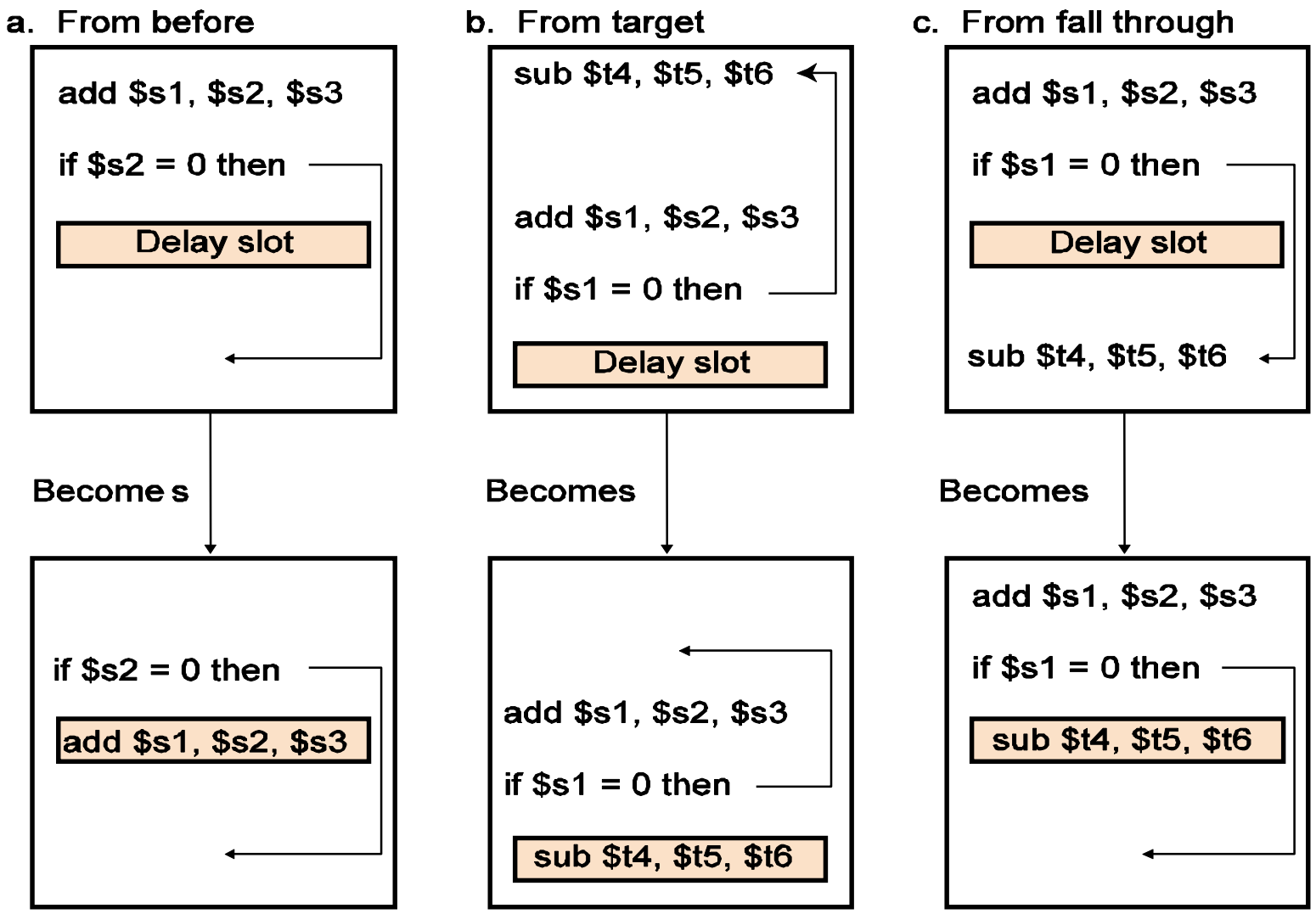
© 2003 Elsevier Science (USA). All rights reserved.

Control Hazards - Solutions

- **Delayed branches – code rearranged by compiler to place independent instruction after every branch (in delay slot).**

```
add  $R4,$R5,$R6          beq  $R1,$R2,20
beq  $R1,$R2,20     ➡     add  $R4,$R5,$R6
lw $R3,400($R0)           lw $R3,400($R0)
```

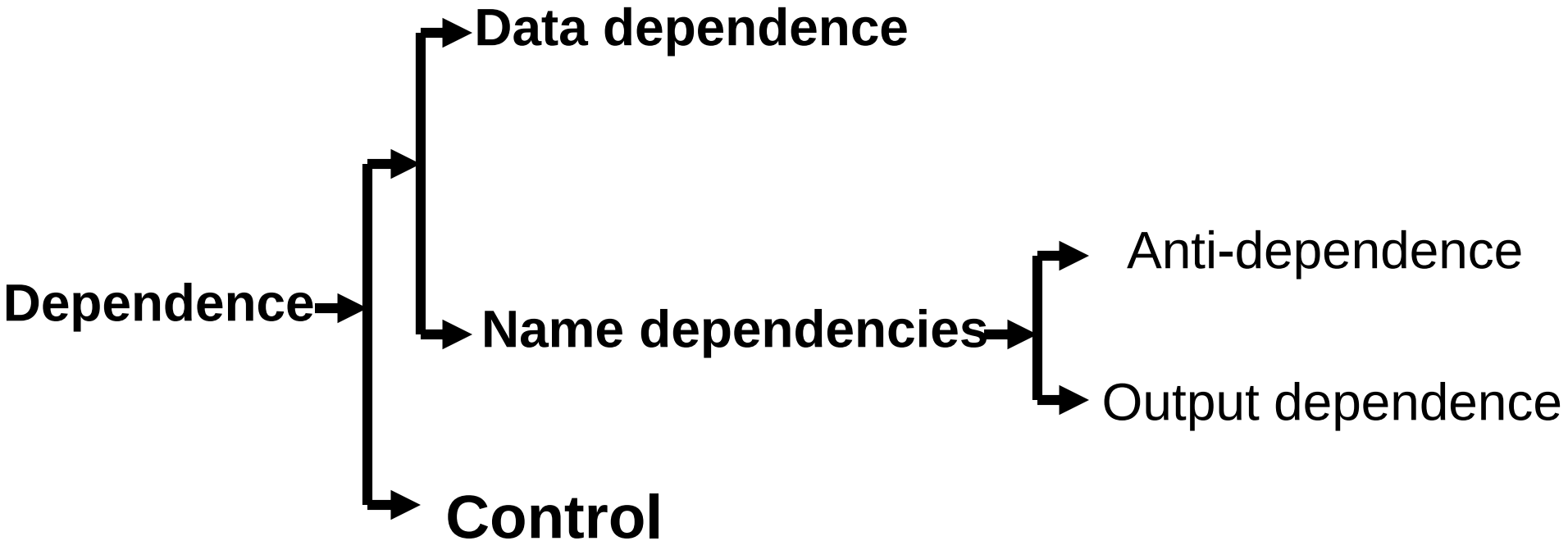
Scheduling the Delay Slot



Summary - Control Hazard Solutions

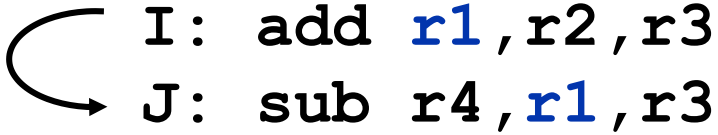
- **Stall** - stop fetching instr. until result is available
  - Significant performance penalty
  - Hardware required to stall
- **Predict** - assume an outcome and continue fetching (undo if prediction is wrong)
  - Performance penalty only when guess wrong
  - Hardware required to "squash" instructions
- **Delayed branch** - specify in architecture that following instruction is always executed
  - Compiler re-orders instructions into delay slot
  - Insert "NOP" (no-op) operations when can't use (~50%)
  - This is how original MIPS worked

Dependencies



Data Dependence and Hazards

- $Instr_J$ is data dependent on $Instr_I$
  $Instr_J$ tries to read operand before $Instr_I$ writes it



- or $Instr_J$ is data dependent on $Instr_K$ which is dependent on $Instr_I$
- Caused by a "True Dependence" (compiler term)
- If true dependence caused a hazard in the pipeline, called a Read After Write (RAW) hazard

Data Dependences through registers/memory

- **Dependences through registers are easy:**

  lw r10,10(r11)

  add r12,r10,r8

  just compare register names.

- **Dependences through memory are harder:**

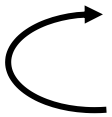
  sw r10,4 (r2)

  lw r6,0(r4)

  is r2+4 = r4+0? If so they are dependent, if not, they are not.

Name Dependence #1: Anti-dependence

- **Name dependence: when 2 instructions use the same register or memory location, called a name, but no flow of data between the instructions is associated with that name; 2 versions of name dependence**
- **$Instr_J$ writes operand *before* $Instr_I$ reads it**

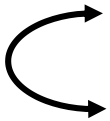


```
I: sub r4,r1,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- **Called an "anti-dependence" by compiler writers. This results from reuse of the name "r1"**
- **If anti-dependence caused a hazard in the pipeline, called a Write After Read (WAR) hazard**

Name Dependence #2: Output dependence

- Instr$_J$ writes operand ***before*** Instr$_I$ writes it.



```
I: sub r1,r4,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- Called an "output dependence" by compiler writers
  This also results from the reuse of name "r1"
- If anti-dependence caused a hazard in the pipeline, called a Write After Write (WAW) hazard

Dependences and hazards

- Dependences are a property of programs.
- If two instructions are data dependent they cannot execute simultaneously.
- Whether a dependence results in a hazard and whether that hazard actually causes a stall are properties of the pipeline organization.
- Data dependences may occur through registers or memory.

Instruction Dependence Example

- **For the following code identify all data and name dependence between instructions and give the dependency graph**

```
1    L.D     F0, 0 (R1)
2    ADD.D   F4, F0, F2
3    S.D     F4, 0(R1)
4    L.D     F0, -8(R1)
5    ADD.D   F4, F0, F2
6    S.D     F4, -8(R1)
```

Instruction Dependence Example

- **For the following code identify all data and name dependence between instructions and give the dependency graph**

```
1   L.D     F0, 0 (R1)
2   ADD.D   F4, F0, F2
3   S.D     F4, 0(R1)
4   L.D     F0, -8(R1)
5   ADD.D   F4, F0, F2
6   S.D     F4, -8(R1)
```

True Data Dependence:

Instruction 2 depends on instruction 1 (instruction 1 result in F0 used by instruction 2), Similarly, instructions (4,5)
Instruction 3 depends on instruction 2 (instruction 2 result in F4 used by instruction 3), Similarly, instructions (5,6)

Name Dependence:

Output Name Dependence (WAW):

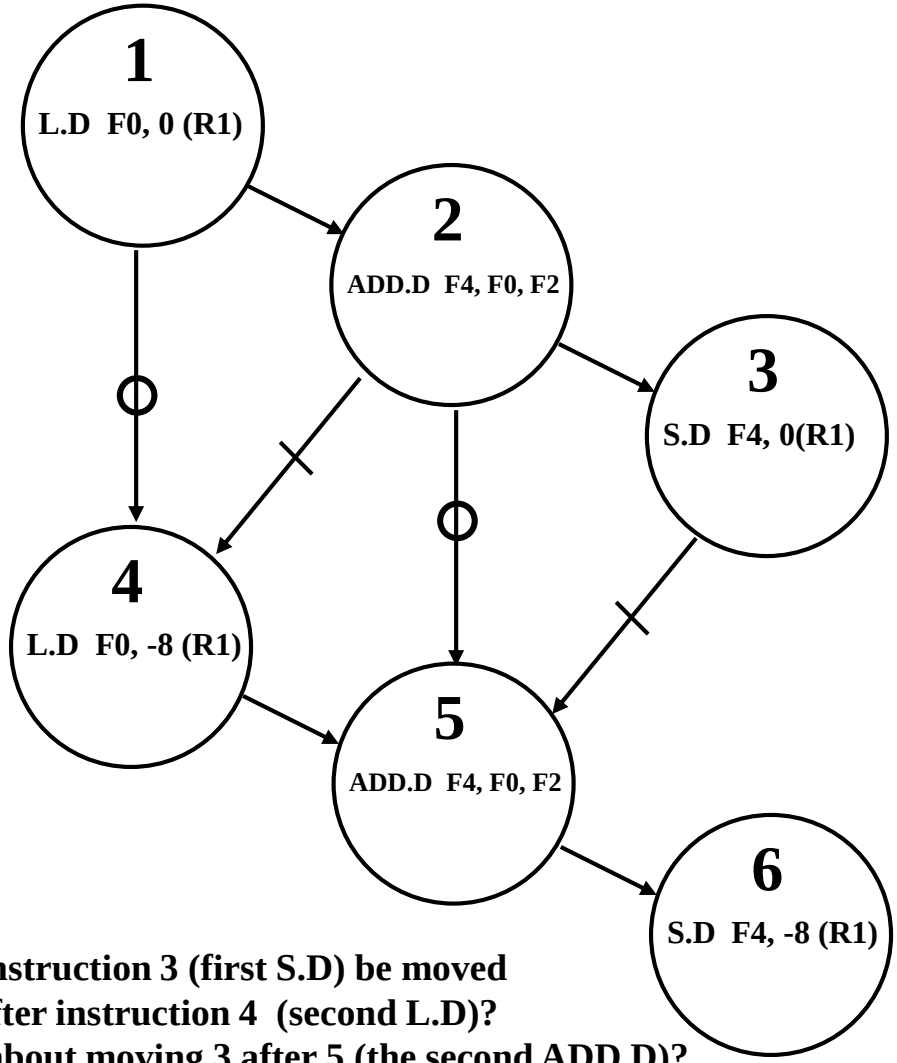
Instruction 1 has an output name dependence over result register (name) F0 with instructions 4
Instruction 2 has an output name dependence over result register (name) F4 with instructions 5

Anti-dependence (WAR):

Instruction 2 has an anti-dependence with instruction 4 over register (name) F0 which is an operand of instruction 1 and the result of instruction 4
Instruction 3 has an anti-dependence with instruction 5 over register (name) F4 which is an operand of instruction 3 and the result of instruction 5

Instruction Dependence Example

Dependency Graph



Example Code

1	L.D	F0, 0 (R1)
2	ADD.D	F4, F0, F2
3	S.D	F4, 0(R1)
4	L.D	F0, -8(R1)
5	ADD.D	F4, F0, F2
6	S.D	F4, -8(R1)

Date Dependence:
(1, 2) (2, 3) (4, 5) (5, 6)

Output Dependence:
(1, 4) (2, 5)

Anti-dependence:
(2, 4) (3, 5)

Can instruction 3 (first S.D) be moved just after instruction 4 (second L.D)? How about moving 3 after 5 (the second ADD.D)? If not what dependencies are violated?

Can instruction 4 (second L.D) be moved just after instruction 1 (first L.D)? If not what dependencies are violated?

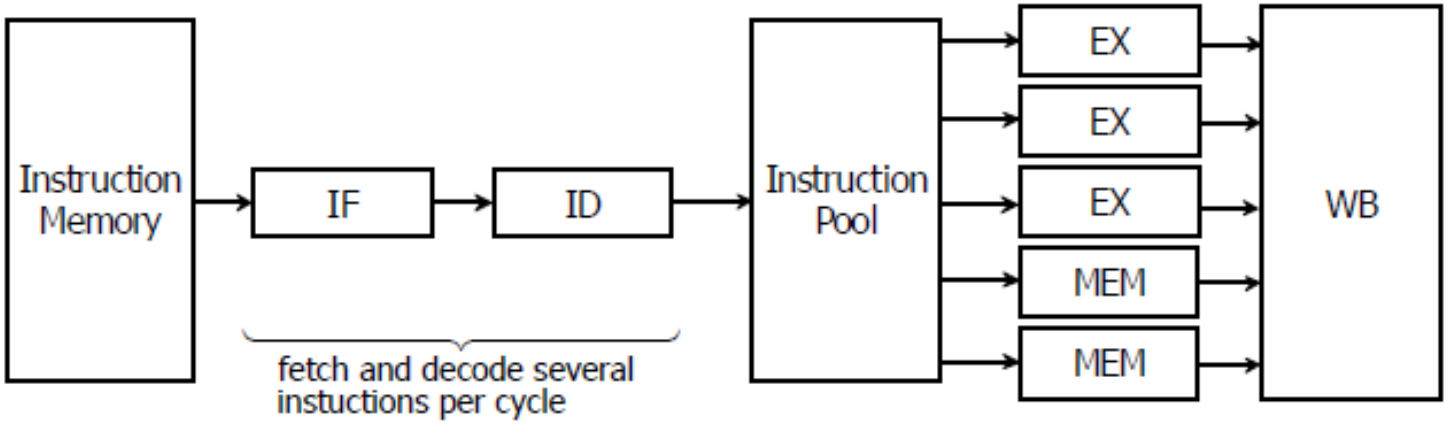
Dynamic vs. Static Scheduling

- Data hazards in a program cause a processor to stall.
- With static scheduling the compiler tries to reorder these instructions during compile time to reduce pipeline stalls.
  - Uses less hardware
  - Can use more powerful algorithms
- With dynamic scheduling the hardware tries to rearrange the instructions during run-time to reduce pipeline stalls.
  - Simpler compiler
  - Handles dependencies not known at compile time
  - Allows code compiled for a different machine to run efficiently.

ILP and Data Hazards

- HW/SW must preserve program order: the order in which instructions would execute in if executed sequentially 1 at a time as determined by original source program
- HW/SW goal: exploit parallelism by preserving program order only where it affects the outcome of the program
- Instructions involved in a name dependence can execute simultaneously if name used in instructions is changed so instructions do not conflict
  - Register renaming resolves name dependence for registers
  - Either by compiler or by HW

Instruction Pool Example



			CC	Utilization
I1 I2 I3	ADDI R1, R0, #10 ADDI R2, R0, #87 LW R6, 100(R4)	No dependencies Run on 2 EX + 1 MEM	1	3
I4	ADD R4, R2, R6	Dependency — runs in EX	2	1
I5	ADD R7, R4, R6	Dependency — runs in EX	3	1
I6	SW 100(R4), R7	Dependency — runs in MEM	4	1
I7	SW 200(R0), R8	1 store per CC — runs in MEM	5	1

I7 has no dependency — could **RESCHEDULE** to CC1, CC2, CC3

Instruction Rescheduling Strategies

Instruction dependencies

- RAW hazard — instruction I_{k+j} reads operand before I_k writes it
- WAW hazard — instruction I_{k+j} writes operand before I_k writes it
- WAR hazard — instruction I_{k+j} writes operand before I_k reads it

Instruction dependency stall

- Delay I_{k+j} until I_k completes

Register renaming

- Allocate separate register sets to independent instruction blocks
- Eliminates dependencies between independent blocks using same registers

Instruction rescheduling

- Move independent instructions between I_k and I_{k+j}
  - Increase instruction "distance" in pipeline to resolve dependence
- Compiler optimization
  - Compiler reschedules object code (machine code listing) in second pass
- Dynamic rescheduling (out-of-order execution)
  - Hardware executes instructions in most efficient order
  - Chooses instructions with no unresolved dependencies for execution

Efficient Register Reuse

Instruction block N

Allocate R1

Overwrite for use as destination operand

Instructions access R1

Access as source and destination operand

De-allocate (**RETIRE**) R1

After final source access in instruction block

Instruction block N+1

Re-allocate R1

Register Reuse	
`LW R1, 0(R2)`	; allocate R1
`ADDI R1, R1, #1`	; R1 is source and destination
`SW 0(R2), R1`	; R1 is source — de-allocate R1
`ADD R1, R2, R3`	; re-allocate of R1

Register Reuse versus Register Renaming

Register renaming

- Instruction block N
  - Allocate `R1`
  - Instructions access `R1`
  - De-allocate (**RETIRE**) `R1`
- Instruction block N+1
  - Allocate new register instead of reuse

Eliminates fictitious data dependence

- Instruction blocks use independent registers
- Enables rescheduling

Example

Reuse	Renaming	Rescheduling
`LW R1, 0(R2)`	`LW R1, 0(R2)`	`LW R1, 0(R2)`
`ADDI R1, R1, #1`	`ADDI R1, R1, #1`	`ADD R10, R2, R3`
`SW 0(R2), R1`	`SW 0(R2), R1`	`ADDI R1, R1, #1`
`ADD R1, R2, R3`	`ADD R10, R2, R3`	`SW 0(R2), R1`

Instruction Rescheduling for DLX

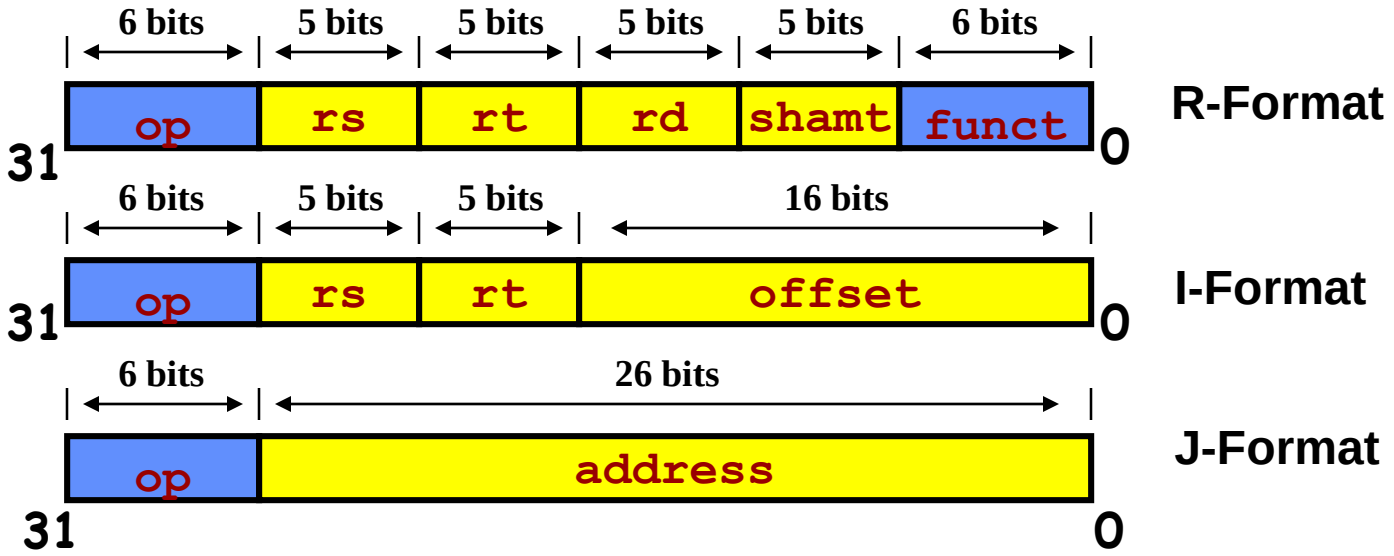
Compiler reschedules instruction issue

- Respect real data flow dependencies
- Eliminate fictitious data dependencies by register renaming
- Enhance efficiency
  - Insert independent work between dependent instructions

High Level Code	First Pass Assembly Code	Rescheduled Code for DLX
X = X + 123	LW R2,[X] ADD R2,R2,#123 SW [X],R2	LW R2,[X] LW R3,[Y] ; Rename R2 LW R4,[Z] ; Rename R2
Y = Y - 456	LW R2,[Y] ; Reuse R2 SUB R2,R2,#456 SW [Y],R2	ADD R2,R2,#123 SUB R3,R3,#456 SUB R4,R4,#789
Z = Z - 789	LW R2,[Z] ; Reuse R2 SUB R2,R2,#789 SW [Z],R2	SW [X],R2 SW [Y],R3 SW [Z],R4

MIPS Instructions

- All instructions exactly 32 bits wide
- Different formats for different purposes
- Similarities in formats ease implementation



MIPS Instruction Types

- **Arithmetic & Logical** - manipulate data in registers

```
add $s1, $s2, $s3     $s1 = $s2 + $s3
or $s3, $s4, $s5      $s3 = $s4 OR $s5
```

- **Data Transfer** - move register data to/from memory

```
lw $s1, 100($s2)      $s1 = Memory[$s2 + 100]
sw $s1, 100($s2)      Memory[$s2 + 100] = $s1
```

- **Branch** - alter program flow

```
beq $s1, $s2, 25      if ($s1==$s1) PC = PC + 4 + 4*25
```